

Computing in a Safe House

Accountable Universally Composable Asynchronous Secure Distributed Computing

PIERRE CIVIT, EPFL (École Polytechnique Fédérale de Lausanne), Switzerland

DANIEL COLLINS, New York University and The Hebrew University, USA and Israel

VINCENT GRAMOLI, University of Sydney and Redbelly Network, Australia

RACHID GUERRAOUI, EPFL (École Polytechnique Fédérale de Lausanne), Switzerland

JOVAN KOMATOVIC, Category Labs, USA

MANUEL VIDIGUEIRA, Chainlink Labs, USA

POURIYA ZARBAFIAN, Amaroo, Australia

In non-synchronous networks, partitioning arguments show that t -resilient protocols among n processes could violate safety when the number of malicious processes f is $\geq n - 2t$. This fragility motivates augmenting such protocols with accountability schemes to deter safety violations. So far however, such schemes have been limited in their verifiability, scalability or privacy.

This paper presents τ_{zk-scr} , a universal compiler that circumvents such limitations. The compiler transforms any protocol $\mathcal{P}$, that is secure against semi-honest crash-failure adversaries, into a Byzantine-tolerant, accountable counterpart $\tilde{\mathcal{P}}$. Essentially, we devise τ_{zk-scr} by deconstructing the celebrated CLOS compiler (STOC 2002), observing that each resulting component is “easily accountable”, and globally propagating the accountability through the reconstruction. The guarantees provided by τ_{zk-scr} are defined with respect to a resilience threshold $t_\epsilon = \lceil n(\frac{1}{3} - \epsilon) \rceil - 1$, for any $\epsilon \geq 0$. $\tilde{\mathcal{P}}$ preserves the hyperproperties of $\mathcal{P}$, including privacy, input-independence, correctness, and output delivery, whenever $f \leq t_\epsilon$.

If $f > t_\epsilon$, then either: (1) $\tilde{\mathcal{P}}$ emulates $\mathcal{P}$, in the sense that all its hypersafety properties are preserved, though output delivery may not occur; or (2) all correct processes obtain externally verifiable proofs of misbehavior involving a significant subset of faulty parties. By adjusting its parameters, τ_{zk-scr} achieves various trade-offs. Assuming a transparent setup, for any strictly positive constant $\epsilon \in \Omega(1)$, the most efficient instantiation provides security against a 1-delayed-adaptive adversary (i.e., where corruption decisions are postponed just long enough to allow messages in transit to be delivered) with $o(n^2)$ multiplicative communication overhead.

Our results are formalized and proven following the Accountable Universal Composability (AUC) blueprint (S&P 2023), an extension of UC designed to support modular analysis of accountability guarantees.

CONTENTS

Contents	2
1 Introduction	3
2 Overview	6
2.1 Model Overview	6
2.2 Compiler Overview	9
2.3 Applications	11
3 Conclusion	12
4 Summary of the notations	13
5 The Compiler	14
5.1 Specification of $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$	14
5.2 Specification of $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$	16
5.3 Use of $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$	16
5.4 Abstract Machines for Commit&Prove	17
5.5 The Compiler $\tau_{\text{zk-scr}}$	19
A Commit&Prove: Realizing $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ in the $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$ -hybrid model	25
A.1 Non-interactive Commitment	25
A.2 Specification of $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$	25
A.3 Pseudocode Description of $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$'s Realization	25
A.4 Adaptively Secure Commit-and-Prove	29
B Coin-Tossing: Realizing $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$ in the $(\mathcal{F}_{\text{rbc}}^{\text{acc}}, \mathcal{F}_{\text{hacss}})$ -hybrid model	31
B.1 $\mathcal{F}_{\text{hacss}}$: High-Threshold Asynchronous Complete Secret Sharing	32
B.2 Pseudocode Description of $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$'s Realization	33
C ZKPoK: realizing $\mathcal{F}_{\text{zkPoK}}^{1:\text{M}}$	36
C.1 Universally Composable Zero-knowledge Proof-of-Knowledge	36
C.2 Achieving One-to-Many ZKPoK	37
D The public judge and the judicial functionality	39
References	41

1 INTRODUCTION

MPC feasibility. In secure multi-party computation (MPC) [67, 75], a set Ψ of n processes, $p_1, \dots, p_n$, collectively emulate a specified functionality (e.g., distributed machine learning, voting, auctions, payment systems), while tolerating adversaries ranging from honest-but-curious to inducing arbitrary malicious faults. Even in fully asynchronous systems, any functionality computable by a trusted third party can be emulated while tolerating up to $n/3$ malicious faults [22] or even $n/2$ non-equivocating malicious faults [9].¹ However, partitioning arguments [28] can be used to explain why many functionalities become impossible to compute if message delivery suffers arbitrary delays and the number of arbitrary malicious faults f is at least $n/3$. This fragility does not only concern liveness. Even the safety of those functionalities can be violated if $f \geq n - 2t$, where t is the resilience threshold of the protocol.

“Easy” accountability. To address this weakness, researchers have explored the concept of *accountability* [97]. In accountable protocols, violations of critical properties (e.g., agreement in a blockchain) result in the detection and non-repudiable attribution of misbehavior to specific malicious processes.

A recent line of work has focused on the appealing goal of making classic agreement primitives accountable: if safety is violated due to an over-corrupted system, correct processes should be able to identify and publicly expose the culprits. In particular, the *ABC*-like approach [53, 55, 56] introduces the *accountable confirmer*: a simple, low-overhead, and plug-and-play functionality denoted $\mathcal{F}_{\text{conf}}^{\text{acc}}$. In a nutshell, the accountable confirmer functionality instructs processes to ratify a (pre-)agreement by attaching non-repudiable signatures before deciding on a value. In the nominal case, i.e., when $f \leq t$, this adds only a single round of latency to the original protocol. In the degraded case ($f > t$), however, any disagreement is eventually traced back to a significant set of faulty processes that signed conflicting ratification messages for different values. The confirmer can be sequentially composed with any easily accountable agreement functionality $\mathcal{F}_{\text{ea}}$, such as Byzantine Agreement $\mathcal{F}_{\text{ba}}$ (for any validity condition), Reliable Broadcast $\mathcal{F}_{\text{rbc}}$, Consistent Broadcast $\mathcal{F}_{\text{cbc}}$, or one-to-many zero-knowledge proof of knowledge $\mathcal{F}_{\text{zkPoK}}^{1:M}$, to yield its accountable counterpart $\mathcal{F}_{\text{ea}}^{\text{acc}}$. These accountable functionalities extend the guarantees of each agreement primitive by ensuring that, in case of a disagreement caused by excessive corruption, all honest processes eventually obtain externally verifiable evidence identifying a significant set of faulty parties. As a result, any protocol whose security reduces to such easily accountable agreement functionalities can itself be compiled into an accountable version with minimal overhead. This includes a broad class of systems, such as distributed ledgers [5], state machine replication [80, 81], bulletin board services [87, 96], and payment systems [32, 65, 89]. While lattice agreement falls outside the scope of the *ABC*-style transformation, this task has also been shown to admit an efficient accountable implementation [72].

“General” accountability. Beyond such task-specific transformations, generic compilers for accountability have also been proposed. Perhaps the most influential work in this direction is PeerReview [90]. In this approach, each process is required to maintain a tamper-evident log that serves as non-repudiable evidence of its behavior. This log must satisfy two key properties: *uniqueness*, meaning that a process cannot plausibly claim multiple inconsistent behaviors; and *soundness*, meaning that the recorded behavior must convincingly demonstrate compliance with the prescribed protocol. However, PeerReview was designed around the notion of auditing, where processes may

¹The flaw identified in [9] by [23] can be resolved by employing an $n/2$ -resilient agreement on a core set (ACS) protocol [66, 95, 99] that avoids reduction to binary agreement with strong unanimity, as the latter cannot be implemented, even with the aid of a trusted party [23]. This observation similarly applies to [61].

be challenged at arbitrary times. If a process fails to respond to a challenge, it becomes permanently suspected by all other participants. This notion is akin to the concept of completeness in the parlance of the failure-detector paradigm [77]: faults eventually yield to permanent suspicion, but not necessarily with irrefutable evidence. In particular, other processes cannot produce externally verifiable proofs of misbehavior [97].

The idea of maintaining a unique and sound execution log echoes the classic reduction [6, 58, 60, 92, 93] commonly attributed to Bracha [26, 27]. This reduction simulates crash-resilient protocols in the presence of Byzantine faults by routing all communication through a reliable broadcast functionality $\mathcal{F}_{\text{rbc}}$. In this setting, uniqueness is enforced via the agreement property of $\mathcal{F}_{\text{rbc}}$, while soundness is guaranteed by having each process verify the execution of others based on plaintext communication, i.e., an approach unconcerned with privacy. Viewed through this lens, the design of the more recent τ_{scr} compiler [57] becomes easier to interpret. The τ_{scr} solution can be understood as a Bracha-style transformation in which the reliable broadcast functionality $\mathcal{F}_{\text{rbc}}$ is replaced by its accountable counterpart $\mathcal{F}_{\text{rbc}}^{\text{acc}}$. The result is a simulation in which either the crash-failure protocol is faithfully emulated, or a breach in $\mathcal{F}_{\text{rbc}}^{\text{acc}}$ occurs. If the breach affects liveness, then the liveness of the original crash-failure protocol is also compromised. A breach that affects safety must originate from a violation of agreement within $\mathcal{F}_{\text{rbc}}^{\text{acc}}$. In that case, all correct processes eventually receive irrefutable proofs of misbehavior. In short, τ_{scr} strengthens the PeerReview approach by upgrading suspicion-based detection to externally-verifiable accountability. Moreover, this decomposition highlights that τ_{scr} can benefit from the recent scalable realization of the accountable confirmer functionality $\mathcal{F}_{\text{conf}}^{\text{acc}}$ [53]. When composed sequentially with a scalable realization of reliable broadcast $\mathcal{F}_{\text{rbc}}$ [25, §3.2], this yields a subquadratic realization of the accountable broadcast functionality $\mathcal{F}_{\text{rbc}}^{\text{acc}}$, and consequently, a transformation with subquadratic multiplicative communication overhead.

Although τ_{scr} significantly broadens the applicability of accountability, it shares a critical limitation with PeerReview and its successor CSAR [10]: none of these approaches preserve privacy. As they rely on simulating the internal behavior of each process (e.g., for validation), they are incompatible with privacy-sensitive protocols, such as agreement protocols that implement common coins via secret sharing [69, 73], or protocols that require secrecy. This limitation raises the natural question: *can we design an accountability compiler that is both privacy-preserving and communication-efficient?*

This paper answers this question in the affirmative. We present such a compiler which we formalize and prove correct in the universal composability (UC) framework [34], offering strong composability guarantees. Our results are contrasted with prior work in Table 1.

Contribution. We present $\tau_{\text{zk-scr}}$, a privacy-preserving compiler that transforms any protocol $\mathcal{P}$, designed to tolerate semi-honest crash-failure adversaries, into an accountable counterpart $\tilde{\mathcal{P}} = \tau_{\text{zk-scr}}(\mathcal{P})$ that withstands active adversaries, capable of arbitrary deviations from the protocol. The transformation reduces to two key functionalities: (1) the accountable counterpart $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ of the one-to-many commit-and-prove functionality introduced in the CLOS compiler [44], and (2) the accountable one-to-many leader-based coin-tossing functionality $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$, invoked once per process (as a leader) during the offline phase, independently of the specific underlying protocol $\mathcal{P}$. Next, we fix $\epsilon \geq 0$ as the resilience slack, which defines the tolerated fault threshold $t_\epsilon = \lceil n(\frac{1}{3} - \epsilon) \rceil - 1$. Our main contribution can be summarised by the following theorem which captures accountability through a globally available functionality $\mathcal{J}$ (formally introduced later), providing the interface for generating and verifying proofs of misbehavior; essentially, $\mathcal{J}$ reduces to a PKI. We stress that the compiler makes no security or threshold assumption on the base protocol $\mathcal{P}$; Theorem 1.1 is purely an emulation statement, preserving whatever (hyper)properties $\mathcal{P}$ itself satisfies, independently of whether $\mathcal{P}$ remains meaningful or secure beyond a given corruption level.

THEOREM 1.1. *Let $\mathcal{P}$ be a distributed protocol. Let $\bar{\mathcal{P}} = \tau_{zk\text{-}scr}(\mathcal{P})$ denote the compiled protocol, operating in the $(\mathcal{F}_{cp}^{1:M,acc}, \mathcal{F}_{lb\text{-}coin}^{1:M,acc})$ -hybrid model (which includes $\mathcal{J}$). For every active adversary $\mathcal{A}$ of $\bar{\mathcal{P}}$, there exists a semi-honest crash-failure adversary/simulator $\mathcal{S}$ for $(\mathcal{P} \mid \mathcal{J})$, such that:*

- **Nominal mode** As long as $f \leq t_\epsilon$, $(\mathcal{A} \mid \bar{\mathcal{P}})$ and $(\mathcal{S} \mid \mathcal{P} \mid \mathcal{J})$ are indistinguishable, i.e. $\bar{\mathcal{P}}$ maintains all the hyperproperties of $\mathcal{P}$.
- **Degraded mode.** If $f > t_\epsilon$, then either:
 - (1) $\bar{\mathcal{P}}$ preserves all hypersafety properties of $\mathcal{P}$ (liveness need not hold), or
 - (2) All correct processes eventually obtain externally verifiable proofs of misbehavior, exposing at least d dishonest parties, where d is the accountability parameter of $(\mathcal{F}_{cp}^{1:M,acc}, \mathcal{F}_{lb\text{-}coin}^{1:M,acc})$.
- **Complexity preservation.** Let rc and mc denote the round and message complexities of the realization of $\mathcal{F}_{cp}^{1:M,acc}$. Each message of $\mathcal{P}$ induces mc messages in $\bar{\mathcal{P}}$, and, in the nominal mode, event-related round complexities are multiplied by a factor rc .

Roughly speaking, our compiler can be viewed as a “deconstruction” of the CLOS compiler [44] followed by its “accountable reconstruction”. Our fundamental observation is that the components that result from the deconstruction are interestingly easily accountable agreement primitives. As a result, when reconstructing the compiler, the universally composable $\mathcal{ABC}$ lifting propagates accountability through the entire compilation.

Full realization. The access to the ideal functionalities $\mathcal{F}_{lb\text{-}coin}^{1:M,acc}$ and $\mathcal{F}_{cp}^{1:M,acc}$ can be replaced by their UC realizations, thanks to the standard substitutability guarantees of the UC framework. We propose a simple UC realization of $\mathcal{F}_{lb\text{-}coin}^{1:M,acc}$ in the $(\mathcal{F}_{rbc}^{acc}, \mathcal{F}_{hacss})$ -hybrid model, where $\mathcal{F}_{hacss}$ is a high-threshold asynchronous complete secret sharing functionality with strong privacy guarantees, than can be implemented via [71, §7]. This realization is conditional on $f \leq t_{acc}^{lb\text{-}coin} = n - t_\epsilon - 1^2$. Additionally, we provide a CLOS-like UC realization of $\mathcal{F}_{cp}^{1:M,acc}$ in the $\mathcal{F}_{zkPoK}^{1:M,acc}$ -hybrid model. Both $\mathcal{F}_{zkPoK}^{1:M,acc}$ and $\mathcal{F}_{rbc}^{acc}$ can be instantiated in the UC framework under a certification (PKI) functionality $\mathcal{F}_{cert}$ [35], by applying the $\mathcal{ABC}$ transformation to existing realizations of their non-accountable counterparts $\mathcal{F}_{zkPoK}^{1:M}$ and $\mathcal{F}_{rbc}$. This results in two instantiations, each with its own trade-offs:

- $(rc, mc, \epsilon, t_{acc}, d) \in \left(\{4\}, \Theta(n\lambda), \Theta(1), \frac{n}{1-\Theta(1)}, \Theta(\lambda) \right)$, with λ a statistical security parameter, assuming a VRF setup and a 1-delayed-adaptive adversary [103].
- $(rc, mc, \epsilon, t_{acc}, d) = (\{4\}, \Theta(n^2), 0, n, \lceil n/3 \rceil)$, offering security even against a strongly adaptive adversary with after-the-fact removal capabilities [1].

Use case. Thanks to its universality, our compiler $\tau_{zk\text{-}scr}$ can be applied to any (potentially light-weight) protocol designed to efficiently realize a particular functionality, rather than just a (costly) general asynchronous MPC (AMPC) protocol intended to handle arbitrary functionalities. That said, to provide concrete and immediately usable illustrations, we apply our transformation to two existing AMPC protocols [9, 61] that tolerate up to $n/2$ non-equivocating corruptions, namely adversaries that may behave arbitrarily but never send conflicting messages. The transformation yields AMPC protocols that tolerate full Byzantine faults up to the optimal (resp., near-optimal) bound t_ϵ for $\epsilon = 0$ (resp., $\epsilon \in \Theta(1)$), and in the regime $f \in (t_\epsilon, n/2)$, either preserves all hypersafety properties or enables each correct process to obtain undeniable, externally verifiable evidence of misbehavior.

²If it is unacceptable to tolerate both (1) $t_{acc} < n - t_\epsilon$ and (2) the possibility that a semi-honest adversary biases the random tapes of the corrupted process in the compiled protocol, then $\mathcal{F}_{lb\text{-}coin}^{1:M,acc}$ could be realized during a synchronous offline setup phase. This can be done using a synchronous accountable MPC protocol [112], in the absence of a trusted entropy source such as the hash of a fresh blockchain block.

Table 1. Comparison of accountable solutions.

Compiler transformations	$\tau_{zk-scr} + [53]$	$[57] + [53]$	$[10, 90]$
Externally-verifiable proofs	✓	✓	✗
$\times o(n^2)$ comm. vs. adaptive $\mathcal{A}$	✓	✓	✗
UC-style security proof	✓	✗	✗
Privacy-preserving	✓	✗	✗
Accountable MPC protocols	$\tau_{zk-scr}([9, 61])$	$[9, 61]$	$[112]$
“Accountability” for $f \in (n/3, n/2)$	✓	✓	✓
Tolerate equivocation	✓	✗	✓
Tolerate asynchrony	✓	✓	✗

ID-MPC. In *synchrony*, fairness is impossible below $n/2$ honest parties [59], motivating MPC with identifiable abort [7, 8, 17–19, 52, 62–64, 68, 94, 115] and culminating in publicly accountable synchronous MPC [112]: full guarantees for $f < n/2$, else identification of at least $\lceil n/2 \rceil$ faulty parties. This critically relies on synchrony (unbounded silence is fault); we work *asynchronously*.

Paper organization. In Section 2, we overview the model and outline our construction. In Section 5, we reduce the τ_{zk-scr} compiler to the ideal functionalities $\mathcal{F}_{cp}^{1:M,acc}$ and $\mathcal{F}_{lb-coin}^{1:M,acc}$, with no computational assumption. We then provide the realization of $\mathcal{F}_{cp}^{1:M,acc}$ in the $\mathcal{F}_{zkPoK}^{1:M,acc}$ -hybrid model in Appendix A, and $\mathcal{F}_{lb-coin}^{1:M,acc}$ in the $(\mathcal{F}_{rbc}^{acc}, \mathcal{F}_{haccs})$ -hybrid model in Appendix B. Appendix C explains how to realize the $\mathcal{F}_{zkPoK}^{1:M}$ functionality (prior to its $\mathcal{ABC}$ -based transformation used to obtain $\mathcal{F}_{zkPoK}^{1:M,acc}$). Figure 1, page 12, depicts a visual roadmap.

2 OVERVIEW

The goal of this section is to offer a clear high-level description of our contribution before delving into all technical details. Section 2.1 outlines the general framework for accountable universal composability. Section 2.2 gives a concise description of our compiler, τ_{zk-scr} , assuming access to the functionalities $\mathcal{F}_{cp}^{1:M,acc}$ and $\mathcal{F}_{lb-coin}^{1:M,acc}$. Their concrete realizations are detailed in Appendix A and Appendix B, respectively.

2.1 Model Overview

Universal composability. Universal composability (UC) [37, 98] is a widely used framework for analyzing cryptographic protocols through simulation-based security because of its first-class support for composability.

The framework models a distributed system as a collection of (probabilistic) Interactive Machines (IMs). Three types of IM systems are considered: (i) protocol, (ii) adversarial, and (iii) environmental systems. These model (i) real and ideal protocols/functionalities, (ii) adversaries and simulators, and (iii) higher-level protocols, which typically provide inputs to the protocol parties and act as “distinguishers”. The pure non-determinism (e.g., message scheduling) is resolved by a master IM, typically controlled by the environment. In practice, however, the most relevant setting is when the environment indirectly delegates the meaningful scheduling decisions to the adversary. Each fixed master IM induces a probabilistic distribution over executions. The runtime of the global system, as well as its subsystems, is defined such that the overall runtime of an IM system is polynomially

bounded in the security parameter κ plus the length of the (possible) external input, except with negligible probability.

We say that a (concrete) protocol system $\tilde{\mathcal{P}}$ emulates (or realizes) a (more abstract) protocol system $\mathcal{P}$ if, for every allowed adversary $\mathcal{A}$ interacting with $\tilde{\mathcal{P}}$, there exists an allowed adversary $\mathcal{S}$, called simulator, such that, the concurrent interaction $\mathcal{A}|\tilde{\mathcal{P}}$ between $\mathcal{A}$ and $\tilde{\mathcal{P}}$, and the concurrent interaction $\mathcal{S}|\mathcal{P}$ between $\mathcal{S}$ and $\mathcal{P}$, are indistinguishable. This means that, for every environment $\mathcal{E}$ of both $\mathcal{A}|\tilde{\mathcal{P}}$ and $\mathcal{S}|\mathcal{P}$, the probability that $\mathcal{E}$ writes a different decision bit on some dedicated decision tape in $\mathcal{E}|\mathcal{A}|\tilde{\mathcal{P}}$ and $\mathcal{E}|\mathcal{S}|\mathcal{P}$ is negligible in κ (vanishing faster than the inverse of any polynomial in κ), where the probabilities are taken over the random tapes of the IMs in both systems.

Crucially, UC emulation is composable: if $\mathcal{P}$ emulates $\mathcal{F}$, then $\mathcal{P}$ can replace $\mathcal{F}$ as a subroutine in any higher-level protocol without compromising security. In particular, a protocol $\mathcal{P}$ may rely on auxiliary ideal functionalities (e.g., $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$, or $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$), which can be safely replaced by their concrete realizations.

The indistinguishability is proven by establishing a probabilistic forward (no rewinding) simulation from $\mathcal{A}|\tilde{\mathcal{P}}$ to $\mathcal{S}|\mathcal{P}$. While such simulation implies preservation of *hypersafety properties* (i.e., nothing bad happens that wouldn't happen in the abstract world), it does not, in general, capture (*hyper*)*liveness* (i.e., something good eventually happens). This is addressed by requiring that the simulation also preserves *live executions*, here restricted to *fair executions* [11, 12, 15], where any continuously enabled action (e.g., the delivery of a pending message) is eventually triggered within a polynomial number of steps. This approach is an adaptation of classical liveness-preserving refinement based on fair traces inclusion [101]. In our context, this allows us to characterize meaningful live/fair executions in both concrete and abstract systems: (1) At the abstract level, a fair trace satisfies certain post-conditions (e.g., a delayed output delivery is eventually triggered) whenever its corresponding pre-conditions (e.g., enough inputs received) are met. (2) At the fully concrete message-passing level, fair executions include those where all messages between so-far honest parties are eventually delivered (e.g., as in [88]). We then require that every fair execution of the concrete system $\mathcal{A}|\tilde{\mathcal{P}}$ maps via the simulation to a fair execution of the abstract system $\mathcal{S}|\mathcal{P}$. This enables the proof of *liveness-preserving emulation* and justifies the interpretation of $\tilde{\mathcal{P}}$ as faithfully realizing the guarantees of $\mathcal{P}$, not only in terms of (hyper)safety but also (hyper)liveness.

As prescribed by Shoup et al. [29–31, 113, 114], we only consider *efficient protocols*, where computational resources are uniformly bounded. That is, the number of messages sent and the total computation performed by honest parties are bounded by fixed polynomials (independent of the adversary) in the security parameter and the number of parties, with all but negligible probability. This notion of efficiency is composable and rules out pathological protocols that would trivially satisfy liveness by overloading the system, for instance, by triggering a super-polynomial number of events that the master IM cannot process within polynomial time.

Accountable universal composability (AUC). The AUC framework [86] extends UC with explicit accountability requirements. It is built on iUC [33], which instantiates IITM [98]; UC itself embeds into IITM [111]. In IITM/iUC, composition with joint or global state follows directly from the base composition theorems and does not require the GUC/JUC extensions [42, 46]: a globally available resource is modeled by exposing sufficiently many I/O tapes so that the environment and relevant protocols connect to the same instance.

In AUC, functionalities additionally prescribe how security violations (e.g., breaches of agreement) must expose malicious parties. We associate each accountable functionality with the same *public judge*, also called Judge, a protocol locally executable by any process (including external clients), that produces *verdicts* from *publicly verifiable evidence*. We require: (1) *fairness*—honest parties

are never falsely blamed; (2) *external verifiability*—third parties can validate the evidence; and (3) *individual accountability*—verdicts take the form $\text{dis}(D)$, meaning that all processes in D are deemed dishonest. For presentation simplicity, we refer to one globally accessible instance Judge, while any correct party may run a local copy. AUC also features a meta-party *supervisor* to track violations of the *underlying assumptions* (e.g., synchrony breaks), ensuring that the ideal and real worlds remain aligned when such assumptions fail. In our setting, the only assumption we tie to violations is the threshold condition $f \leq t_\epsilon$, which UC-like frameworks already track via standard corruption/status queries; we therefore do not use the supervisor.

Accountable functionalities enrich the adversarial interface with *Break* requests. We consider BreakLiveness and BreakAgreement(π), which affect guarantees only when $f > t_\epsilon$. If BreakLiveness is accepted, the adversary may indefinitely delay output notifications. If BreakAgreement(π) is accepted, the adversary may violate agreement as specified by the functionality, but only if π is a *judicial report* that enables incrimination: concretely, it must allow delivering to each correct party p_i a proof π_i such that, if submitted to the judge, it yields a verdict $\text{dis}(D_i)$ with $|D_i|$ above the functionality’s accountability threshold. Technically, the functionality must also accept BreakAgreement(π) when $f \leq t_\epsilon$ *without* affecting safety or liveness; in this case it merely disseminates the per-party proofs π_i . This models *absurd self-incrimination* (voluntary self-reporting without advantage) that can occur in the real world and must be simulatable.

The generation and checking of judicial reports are handled by an auxiliary functionality $\mathcal{F}_{\text{judi}}$, the multi-signature analogue of the certification functionality $\mathcal{F}_{\text{cert}}$ [36]. All (accountable) realizations are therefore shown in the $\mathcal{J}$ -hybrid model, where $\mathcal{J} = \text{Judge} \mid \mathcal{F}_{\text{judi}}$. This captures external verifiability while keeping the environment interface identical in both ideal and real worlds. The full specification of $\mathcal{J}$ is given in Appendix D.

In our construction, accountability reduces to the accountable agreement of the confirmer $\mathcal{F}_{\text{conf}}^{\text{acc}}$. Intermediate layers merely forward judicial reports (and derived verdicts) upward. We do not present evidence collection at those layers, as it is provided by the concrete realization of $\mathcal{F}_{\text{conf}}^{\text{acc}}$ in [53].

Processes, network, and adversary. We consider a static set of n processes, $\Psi = \{p_1, \dots, p_n\}$, with public IDs, communicating via message-passing. Communication occurs over a point-to-point, reliable, authenticated network modeled by the ideal authenticated network functionality $\mathcal{F}_{\text{auth}}$. This guarantees that (1) every pair of processes can communicate, (2) the recipient can always identify the sender, and (3) a message sent by one process to another is eventually (i.e., in fair executions) delivered. We consider a fully asynchronous network with no upper bound on message delays, although our transformation can also apply to protocols assuming partial synchrony [74], where the network initially behaves asynchronously but becomes synchronous after some unknown Global Stabilization Time (GST). Note that $\mathcal{F}_{\text{auth}}$ does not inherently ensure privacy; this must be addressed within the protocol using appropriate cryptographic algorithms. The adversary interacts with both the network and the processes. The (polynomially bounded) adversary is unable to break the cryptographic primitives (e.g., ZKP or commitment) involved in the realizations of the underlying functionalities introduced later in the protocol. It controls message scheduling, deciding when messages in transit are delivered, and can dynamically corrupt up to $f \in [0 : n]$ processes. We define two thresholds: an *optimistic* threshold $t_\epsilon \in [0, n/3]$, and a *pessimistic* threshold $t_{\text{acc}} \in [0 : n]$. The compiled protocol must securely emulate the original protocol in the *nominal mode* (when $f \leq t_\epsilon$) and ensure accountability in the *degraded mode* (when $t_\epsilon < f \leq t_{\text{acc}}$).

2.2 Compiler Overview

The accountable Bracha transformation. A well-studied approach to simulating crash failures on top of Byzantine failures [6, 26, 27, 58, 60, 92, 93] involves reliable broadcast. We denote by $\mathcal{F}_{\text{rbc}}$ the corresponding ideal functionality. This simulation, often attributed to Bracha [6, 27], acts as a module θ that (1) links the network layer to a crash-resilient protocol $\mathcal{P}$, and (2) only allows messages from processes deemed to be behaving correctly to reach $\mathcal{P}$. In this way, protocol deviations are treated as crash failures. The simulation employs the reliable-broadcast functionality $\mathcal{F}_{\text{rbc}}$ to ensure that all correct processes agree on a common prefix of valid behavior for each process. Correct processes simulate the behavior of others, maintaining the necessary state information, applying updates to simulated states at each step, and validating that new messages are consistent with the simulated behavior before forwarding them to $\mathcal{P}$. Since there exist reliable broadcast implementations [3, 26] that tolerate up to $\lceil n/3 \rceil - 1$ Byzantine failures, this simulation implies that any t^c -resilient protocol for crash faults can be transformed into a protocol that tolerates up to $\min(t^c, \lceil n/3 \rceil - 1)$ Byzantine faults. Of course, no t -resilient implementation of $\mathcal{F}_{\text{rbc}}$ can guarantee agreement if $f > n - 2t$. In this case, we have to be content with accountability. To achieve this additional property, τ_{scr} [57] essentially replaces $\mathcal{F}_{\text{rbc}}$ by its accountable counterpart $\mathcal{F}_{\text{rbc}}^{\text{acc}}$, where a disagreement is eventually followed by the delivery of irrefutable proofs of misbehavior. Simple implementations of accountable reliable broadcast have been proposed in [54–56]. However, this transformation does not preserve privacy properties, since it requires simulating the behavior of each process, which may include sensitive information. For instance, protocols that depend on common coins implemented using secret sharing techniques may fail to ensure liveness if privacy is not maintained. To handle privacy issues, we naturally combine $\mathcal{F}_{\text{rbc}}^{\text{acc}}$ with zero-knowledge techniques, following the “commit&prove” paradigm, formalized in [44].

The GMW-CLOS construction. Introduced in 1987 by Goldreich, Micali, and Wigderson [83, 84], the GMW compiler essentially compels potentially malicious parties to behave as if they were in the “honest-but-curious” model, all while preserving privacy. It achieves this through three core steps:

- (1) In the setup phase, for each process, a random tape is generated, while other parties receive a corresponding commitment.
- (2) Each new input is associated with a commitment that must be published, but that will never be opened.
- (3) Using the zero-knowledge proof-of-knowledge functionality $\mathcal{F}_{\text{zkPoK}}$ [85], parties demonstrate that their actions adhere to the protocol specification, relative to the committed material (i.e., using the committed randomness).

Naturally, to ensure privacy, the functionality $\mathcal{F}_{\text{zkPoK}}$ should be “merged” with the $\mathcal{F}_{\text{rbc}}^{\text{acc}}$ functionality, to be plugged into the Bracha-like compiler. This leads us to specify the *accountable one-to-many ZKPoK* functionality $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$, whose realization is discussed below.

The functionality $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$. The functionality $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$ (see Functionality 3 in Appendix A.2) combines properties of $\mathcal{F}_{\text{zkPoK}}$ and $\mathcal{F}_{\text{rbc}}^{\text{acc}}$. Here, a prover aims to convince verifiers (i.e., all other processes) that he “knows” a witness w for a true NP-statement x , without revealing any additional information. This ensures the following properties:

- **Completeness:** Given a valid statement and witness, the prover can convince all verifiers.
- **Soundness:** No prover can convince any verifier of a false statement.
- **Zero-Knowledge:** The proof reveals nothing beyond the truth of the statement, and in particular does not reveal the prover’s witness.

- **Knowledge Extractability:** For any (possibly malicious) prover algorithm, that convinces the verifier with non-negligible probability, there exists an *extractor* that interacts with this prover and recovers a valid witness for the statement. This property captures the idea that the prover must “know” the corresponding witness [20].

Additionally, for a specific session, $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$ guarantees the properties of accountable reliable broadcast:

- **Agreement & Totality:** If $f \leq t_e$ and a correct verifier is convinced by a statement x , every correct verifier is eventually convinced by the same statement x .
- **Accountability:** If a disagreement occurs and $f \leq t_{\text{acc}}$, every judge instance associated with an honest party eventually returns a verdict exposing a significant set of dishonest parties.

At first glance, one might attempt to adapt τ_{scr} by simply replacing $\mathcal{F}_{\text{rbc}}^{\text{acc}}$ with a zero-knowledge counterpart $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$. However, as discussed in [44], such a naive substitution fails in the UC framework, since UC commitments do not reveal the committed value to the receiver, preventing meaningful NP statements from being formed. To address this, we instead reduce to the accountable commit-and-prove functionality $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$, which can itself be realized in the $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$ -hybrid model, assuming a (concrete) UC-secure commitment scheme. The functionality $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ refers an NP-relation $\mathcal{R}_{\text{coin}[\cdot]}$, (formalized in Section 5.3). Roughly, $\mathcal{R}_{\text{coin}[\cdot]}$ corresponds (not-exactly) to pairs $(\beta, (r^{\text{com}}, \check{\beta}))$, where $\check{\beta}$ is the trace of a genuine behavior fragment β serving as the witness, i.e. the projection over the (potentially encrypted) received/sent messages of β . Additionally, the random tape r used for computing β must be the sum of r^{com} (initially committed by $p = \beta.\text{process}$) and $\text{coin}[p]$, fed by the the functionality $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$. If malicious processes manipulating their randomness is not a concern, this functionality can be ignored. If a trusted setup or natural entropy source (e.g., blockchain block hashes) is unavailable, $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$ can be realized using techniques like Random Secret Draw [45, 73], involving dual-threshold asynchronous secret sharing [4], with strong privacy [71], as discussed in Appendix B.

Since $\mathcal{F}_{\text{zkPoK}}^{1:\text{M}}$ is an easily accountable agreement functionality, its accountable counterpart $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$ can be constructed by sequentially composing it with the accountable confirmer functionality $\mathcal{F}_{\text{conf}}^{\text{acc}}$ [53, Theorem 5]. One can realize $\mathcal{F}_{\text{zkPoK}}^{1:\text{M}}$ by reliably broadcasting UC-secure NIZKPoK [47, 49, 51, 79, 102] through the reliable broadcast functionality $\mathcal{F}_{\text{rbc}}$. By leveraging the player-replaceability paradigm [48, 105, 106], it is known that for any $\epsilon > 0$, both $\mathcal{F}_{\text{rbc}}$ [25] and $\mathcal{F}_{\text{conf}}^{\text{acc}}$ [53] can be realized in the $\mathcal{F}_{\text{lottery}}$ -hybrid model [76] with message complexity $O(n\lambda)$, where each message is $O(\kappa)$ bits. The non-interactive lottery functionality $\mathcal{F}_{\text{lottery}}$ itself can be realized [110, §B.5] in the $(\mathcal{F}_{\text{vrf}}, \mathcal{F}_{\text{CA}})$ -hybrid model, where each process registers its VRF public key via a certification authority $\mathcal{F}_{\text{CA}}$ [35] during the offline phase. Although this construction introduces a small bias, the bias is tightly controlled and preserves the asymptotic guarantees with overwhelming probability. The subquadratic $\mathcal{F}_{\text{conf}}^{\text{acc}}$ ’s realization of [53] assumes a 1-delayed adversary [103], meaning that corruption decisions are delayed just long enough to allow all messages in transit to be delivered. Alternatively, if one is willing to tolerate $\Omega(n^2)$ message complexity, $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$ can be implemented with $\epsilon = 0$, against a strongly adaptive adversary with after-the-fact removal capabilities [1, 2], following a more standard all-to-all communication pattern [55].

Main result. The guarantees of our compiler are formalized by Theorem 1.1 stated in the introduction. Here below, we present an intuition of its proof.

INTUITION. Assume the properties of $\mathcal{F}_{\text{cp}}^{1:M,\text{acc}}$ and $\mathcal{F}_{\text{lb-coin}}^{1:M,\text{acc}}$ hold (which is necessarily the case if $f \leq t_\epsilon$). The simulation strategy follows that of [44, Proposition 9.6]. For any fixed real adversary $\mathcal{A}$, we construct a semi-honest crash-failure ideal adversary (simulator) $\mathcal{S}$ from $\mathcal{A}$. Adversary $\mathcal{S}$ runs the protocol $\mathcal{P}$ while internally simulating an execution of $\tilde{\mathcal{P}}$ for $\mathcal{A}$. The key point is that $\mathcal{S}$ can complete the simulation despite being semi-honest, as $\mathcal{A}$ is forced to send all messages via $\mathcal{F}_{\text{cp}}^{1:M,\text{acc}}$, which verifies their correctness. Thus, $\mathcal{A}$ effectively behaves semi-honestly and can be simulated by $\mathcal{S}$. (While $\mathcal{A}$ is not semi-honest and can send arbitrary messages, invalid messages are ignored by $\mathcal{F}_{\text{cp}}^{1:M,\text{acc}}$ in $\tilde{\mathcal{P}}$ and cause no issues.) Conversely, if the agreement property of $\mathcal{F}_{\text{cp}}^{1:M,\text{acc}}$ or $\mathcal{F}_{\text{lb-coin}}^{1:M,\text{acc}}$ is violated (implying $f > t_\epsilon$), accountability is immediately inherited. $\square$

2.3 Applications

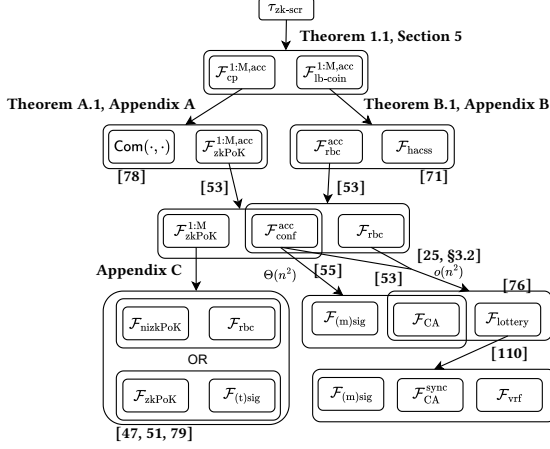
To provide a concrete and immediately usable illustration, we apply our transformation to two existing AMPC protocols that tolerate up to $n/2$ non-equivocating corruptions—that is, adversaries that may behave arbitrarily but never send conflicting messages. The transformation yields AMPC protocols that tolerate fully Byzantine faults up to the optimal (resp., near-optimal) bound t_ϵ for $\epsilon = 0$ (resp., $\epsilon \in \Omega(1)$). In the degraded regime $f \in (t_\epsilon, n/2)$, the new AMPC protocol either preserves all hypersafety properties or enables each correct process to obtain undeniable, externally verifiable evidence of misbehavior.

We first apply the compiler directly to the protocol of [9], which assumes an IND-CPA secure linear homomorphic encryption scheme (e.g., [50]) and an unconditionally hiding, computationally binding linear homomorphic commitment scheme, such as the Pedersen commitment [107]. Although a flaw was later identified in [9] by [23] concerning the agreement on a core set (ACS) subprotocol, this can be remedied by employing a $n/2$ -resilient ACS protocol [66, 95, 99] that does not rely on a reduction to binary agreement with strong unanimity (which is known to be unimplementable in the given setting). This same observation applies to the protocol in [61].

This transformed AMPC protocol can be used to implement the key distribution functionality $\mathcal{F}_{\text{KeyDist}}$ required in the offline phase of [61], yielding its accountable counterpart $\mathcal{F}_{\text{KeyDist}}^{\text{acc}}$. We then apply $\tau_{\text{zk-ser}}$ to the remaining part of the protocol in [61], which additionally assumes a (threshold) fully homomorphic encryption scheme. Due to the composability guarantees of the AUC framework, the subroutine $\mathcal{F}_{\text{KeyDist}}^{\text{acc}}$ can safely be replaced by its realization. The resulting AMPC protocol achieves constant round complexity and $O(n^3 \text{mc})$ message complexity, with each message of size $O(\kappa)$, where again mc refers to the message complexity of the realization of $\mathcal{F}_{\text{cp}}^{1:M,\text{acc}}$.

THEOREM 2.1. *There exists an AMPC protocol $\tilde{\mathcal{P}}_{\text{MPC}}$ for a set of n parties wishing to compute a (possibly randomized) function, represented as an arithmetic circuit C over a finite field and evaluated on a core set of $n - t$ inputs (reflecting the inherent limitations of the asynchronous model), with the following guarantees:*

- **Nominal mode** ($f \leq t_\epsilon$): *The protocol $\tilde{\mathcal{P}}_{\text{MPC}}$ securely computes C , preserving both correctness, privacy, input-independence, and output delivery with $O(1)$ rounds.*
- **Degraded mode** ($f \in (t_\epsilon, n/2)$): *The protocol guarantees either:*
 - *All hypersafety properties of the ideal function evaluation are preserved (though output delivery may not be guaranteed), or*
 - *All correct parties eventually obtain externally verifiable proofs of misbehavior exposing at least d faulty parties, where d is an accountability parameter determined by the realization of $\mathcal{F}_{\text{cp}}^{1:M,\text{acc}}$.*


 Fig. 1. Modularization of τ_{zk-scr} .

3 CONCLUSION

We introduced τ_{zk-scr} (τ_{zk-scr}^{++} when realized with $\mathcal{ABC}^{++}$), a universal compiler that transforms any semi-honest crash-failure-resilient protocol into an accountable, privacy-preserving one resilient to active adversaries. By leveraging $\mathcal{ABC}^{++}$ and its subquadratic realization of $\mathcal{F}_{conf}^{acc}$, our construction overcomes all previous limitations. Specifically, τ_{zk-scr}^{++} :

- (1) Provides a strong notion of externally verifiable individual accountability, ensuring that any security breach results in undeniable proofs of misbehavior against a precisely identified set of dishonest processes, rather than (i) mere permanent suspicion akin to failure-detector-like definitions, or (ii) vague knowledge that some processes in a set must be faulty without pinpointing the culprits.
- (2) Is universally applicable to *any* protocol, regardless of the cryptographic primitives, randomness assumptions, or specific (hyper-)properties such as privacy, fairness, independence of inputs, and correctness that must be preserved.
- (3) Is scalable, inducing only a subquadratic multiplicative communication overhead and a four-fold increase in latency: 3 rounds for the player-replaceable variant of Bracha’s double-echo reliable broadcast [25] and 1 round for the player-replaceable ratifier, with sub-quadratic complexity for each player-replaceable [48] round.
- (4) Maintains its strong guarantees even under (1-delayed) adaptive corruptions, making it robust against bribery attacks where adversaries corrupt processes dynamically during protocol execution.
- (5) Can be directly applied to existing honest-but-curious AMPC protocols to obtain new protocols with unprecedented guarantees: full Byzantine fault tolerance up to the optimal threshold t_e , and strong external accountability in the degraded regime $f \in (t_e, n/2)$.

On the accountability threshold. Our guarantees hold whenever $f \leq t_{acc}$. With only a PKI, $t_{acc} = n - t_e$. Under modest extra randomness setup (a one-shot coin setup or any public entropy beacon), t_{acc} can be raised to $n - \Theta(n)$ while preserving subquadratic overhead, and up to n if one accepts quadratic overhead.

4 SUMMARY OF THE NOTATIONS

Notation	Description
$\Psi = \{p_1, \dots, p_n\}$	Processes, $ \Psi = n$
f	Actual number of corrupted processes
$\epsilon \geq 0$	Small constants to leverage Chernoff bounds
$t_\epsilon = \lceil n(\frac{1}{3} - \epsilon) \rceil - 1$	Optimistic threshold: correctness when $f \leq t_\epsilon$
$t_{acc} = n - \Theta(n)$	Pessimistic threshold: accountability when $f \leq t_{acc}$
λ	Statistical security parameter
κ	Computational security parameter
$\mathcal{F}$	Ideal functionality
$\mathcal{P}$	Real world distributed protocol
$\mathcal{F}^{acc}, \mathcal{F}^{1:M}$	Accountable and one-to-many counterpart of $\mathcal{F}$
$\mathcal{F}_{zkPoK}$	Zero-knowledge proof-of-knowledge functionality
$\mathcal{F}_{cp}$	Commit-and-prove functionality
$\mathcal{F}_{lb-coin}^{1:M}$	One-to-many coin-tossing functionality
$\mathcal{F}_{conf}$	Confirmer functionality
$\mathcal{M}$	Set of messages
$m.sender, m.receiver$	Sender and receiver of a message $m \in \mathcal{M}$
$\mathcal{S}$	Set of states
$s.process$	Process associated with state $s \in \mathcal{S}$
s_i^0	Initial state of process $p_i \in \Psi$
$\mathcal{I} = \mathcal{I}_{msg} \uplus \mathcal{I}_{app}$	Messages and application input notifications
$\mathcal{I}_{adv}$	Adversarial input notifications (e.g., corruptions)
Π_r	Local protocol parametrized by random tape r
$\text{Rand} = \{0, 1\}^{\kappa_r}$	Set of random tapes with $\kappa_r \leq \kappa$
$\mathcal{O}$	Set of output notifications (potentially $\perp$ if absent)
e	Event $e = (r, s, \iota, s', M, o) \in \text{Rand} \times \mathcal{S} \times \mathcal{I} \times 2^M \times \mathcal{O}$
$e.rand, e.old_state, e.new_state$	Random tape, old state, and new state of event e
$e.input, e.sentMsgs, e.output$	Input, set of messages sent, and output of event e
β	Behavior fragment (sequence of events)
$\beta.rand, \beta.process$	Random tape and associated process β
HTFrgs	Set of head-tail behavior of the form $\beta.tail \frown \beta.head$
$\beta.tail$	Tail of a head-tail fragment β : no messages sent
$\beta.head$	Head of a head-tail fragment β
$c := \text{Com}(w, r)$	Commitment of value w with randomness r
$\tilde{\beta} = \text{trace}(\beta)$	trace of $\beta \in \text{HTFrgs}$
HTTraces	Set of head-tail fragment traces
$\mathcal{R}_{coin[\cdot]}$	NP-relation used by $\mathcal{F}_{cp}^{1:M, acc}$

Table 1. Summary of the notations.

5 THE COMPILER

Summary of the notations. We summarize our notations in Table 2, Section 4.

In this section, we present our compiler τ_{zk-scr} , assuming access to two ideal functionalities:

- (1) the *accountable one-to-many commit-and-prove* functionality $\mathcal{F}_{cp}^{1:M,acc}$, and
- (2) the *accountable one-to-many leader-based common-coin* functionality $\mathcal{F}_{lb-coin}^{1:M,acc}$.

Our construction combines ideas from two lines of work: (1) the τ_{scr} compiler [57], itself rooted in the tradition of Bracha-style asynchronous compilers [6, 26, 27, 58, 60, 92, 93], and (2) the GMW/CLOS compiler [44, 83], where a commit-and-prove functionality is used to enforce protocol compliance without revealing auxiliary information.

As noted in [44], a naive adaptation of the GMW compiler to the universally composable (UC) framework is not sound. In particular, simply replacing standard primitives with UC-secure analogs, such as UC commitments, UC coin-tossing, and UC zero-knowledge proofs, fails to yield a secure construction. This is because the receiver of a UC commitment does not learn the committed value; it receives only a formal “receipt” attesting that some value has been committed to. Consequently, there is no well-defined NP-statement that the sender can prove with respect to the commitment “known” by the verifier(s). This contrasts with the original GMW setting, where (perfectly binding) commitments are known explicitly to all parties, allowing meaningful NP relations to be formed. To overcome this issue, the CLOS compiler introduces a dedicated *commit-and-prove* functionality that better models the actual usage of commitments in the GMW protocol. This functionality separates the commitment and proof phases: in the commitment phase, a party commits to values; in the online phase, it can prove arbitrary NP-statements relative to the (ordered) committed values, without revealing anything else. We follow this strategy and introduce an accountable variant of the one-to-many commit-and-prove functionality, namely $\mathcal{F}_{cp}^{1:M,acc}$, which incorporates accountability in the event of safety violation.

Furthermore, to generate the prover’s private random tape used during the compilation, in a way that supports both privacy and correctness, we extend the setup with the accountable leader-based common-coin functionality $\mathcal{F}_{lb-coin}^{1:M,acc}$. The idea mirrors the non-accountable CLOS setup: when a party initiates the commit-and-prove process, it first commits to a private uniformly sampled random value r^{com} , and concurrently acts as the leader of a coin-tossing protocol to independently generate a uniformly sampled public random value r^{coin} . The random tape r used in the proof phase is then defined as $r = r^{com} + r^{coin}$. If either r^{com} or r^{coin} is uniformly distributed and independent of the adversary’s view, then so is r . Moreover, since r^{com} remains secret, the overall proof reveals no information about the witnesses.

This construction yields an accountable analog of the CLOS compiler. Unlike the original CLOS framework, which guarantees only non-triviality (i.e., liveness is ensured for $f = 0$), our compiler tolerates up to t_ϵ faulty processes in the nominal case. We remark that, in the $(\mathcal{F}_{lb-coin}^{1:M,acc}, \mathcal{F}_{cp}^{1:M,acc})$ -hybrid model, the protocol remains perfectly secure against adaptive adversaries, even when the adversary is computationally unbounded.

5.1 Specification of $\mathcal{F}_{cp}^{1:M,acc}$

Similarly to zero-knowledge proofs, the accountable commit-and-prove functionality $\mathcal{F}_{cp}^{1:M,acc}$ is parameterized by a binary relation R . It proceeds in two phases:

- In the *commit phase*, the prover binds itself to a sequence of secret values $(w_1, \dots, w_k)$ by issuing individual commitments, each indexed by its position in the sequence. These commitments can be produced incrementally and in polynomial number.

- In the *prove phase*, the prover submits a statement x and proves, in zero-knowledge, that $R(x, [w_1, \dots, w_k]) = 1$, i.e., that the statement holds relative to the entire ordered sequence of previously committed witnesses.

The formal specification of $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ is provided in Functionality 1. The interaction with the simulator is elided from the pseudocode, but conceptually follows the structure of other accountable functionalities. The simulator controls the scheduling of outputs, and may violate liveness or agreement, but only in degraded mode, i.e. after having corrupted $f > t$ processes. Even in the degraded mode, the simulator cannot violate key properties of the functionality:

- *Soundness*: It cannot cause acceptance of a false statement.
- *Knowledge extractability*: It cannot prove knowledge of a witness it did not send to the functionality.
- *Zero-knowledge*: It cannot learn anything beyond what is revealed by the statements themselves.
- *Binding*: It cannot modify the commitments a posteriori.

If an inconsistency arises (e.g., a successful proof for conflicting statements), this necessarily implies that $f > t_\epsilon$, and a detection mechanism exposes at least d faulty parties through publicly verifiable evidence.

Functionality 1 $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$

Parameters:

$\mathcal{R}$: An NP relation for an NP language.

d : The number of exposed culprits in case of safety violation, when $f > t_\epsilon$

Variables:

$\text{Map}([\text{SID} \times \text{Process}] \rightarrow \text{Array}(\text{Witness})) \ \underline{w}[\]$: witnesses committed by the prover

Events:

request (commit, sid, idx, w): Committing to witness w .

request (prove, sid, idx, x): Proving that x is a valid statement

notification (committed, sid, idx, P): Delivering a commitment from P .

notification (verified, sid, idx, P, x): Verifying the PoK from P w.r.t. x .

Functionality:

Upon receiving (commit, sid, idx, w) from some process P :

If $\underline{w}[\text{sid}, P].\text{length}() = \text{idx}$:

$\underline{w}[\text{sid}, P].\text{append}(w)$

Send (committed, sid, idx, P) to Sim and to all $p_j \in \Psi$ as delayed output.

Else, ignore the message.

Upon receiving (prove, sid, idx, x) from P :

If $\mathcal{R}(x, \underline{w}[\text{sid}, P][:\text{idx}]) = 1$:

Send (verified, sid, idx, P, x) to Sim and to all $p_j \in \Psi$ as delayed output.

// degraded mode ($t_\epsilon < f \leq t_{\text{acc}}$)

Upon receiving (BreakLiveness) from Sim, Sim can indefinitely delay any output (except detection).

Upon receiving (BreakAgreement, sid, P, π) from Sim. If P is corrupted, and $\forall i \in [n]$, the judge, on input $\pi[i]$, returns the (fair) verdict $\text{dis}(D_i)$ with $|D_i| \geq d$, then:

accept any valid request even if conflicting ones were previously accepted, with corresponding notifications of the form (verified, sid, idx, P, x) and (verified, sid, idx, $P, x' \neq x$)

if $f \leq t_{\text{acc}}$: send (detection, sid, $p, D_i, \pi[i]$) to each $p_i \in \Psi$ as delayed output.

► Disagreement

► Accountability

UC realization of $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$. Following the CLOS compiler, $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ can be UC-realized in the $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$ -hybrid model. The underlying idea is that the non-accountable variant $\mathcal{F}_{\text{zkPoK}}^{1:\text{M}}$ is an *easily*

accountable agreement functionality, as defined in [53], and can therefore be upgraded to $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$ via the generic $\mathcal{ABC}$ transformation [53, Theorem 5].

Concretely:

- $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$ is obtained by sequential composition of $\mathcal{F}_{\text{zkPoK}}^{1:\text{M}}$ with the accountable confirmer functionality $\mathcal{F}_{\text{conf}}^{\text{acc}}$, whose UC-realization is detailed in [53, Appendix B].
- $\mathcal{F}_{\text{zkPoK}}^{1:\text{M}}$ can be realized by reliably broadcasting a UC-secure non-interactive ZK proof of knowledge (NIZKPoK) via $\mathcal{F}_{\text{rbc}}$.
- Alternatively, one can rely on interactive ZKPoKs, as in [91]. Both realization strategies are discussed in Appendix C, together with the associated primitive functionalities $\mathcal{F}_{\text{nizkpok}}$ and $\mathcal{F}_{\text{zpkpok}}$.
- $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ is realized in the $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$ -hybrid model in Appendix A as follows:
 - **Commit phase:** The prover commits to a value using an equivocal commitment scheme Com [78], and proves, via $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$, knowledge of the committed value. This yields an accountable “commit-with-knowledge” protocol, enabling the simulator to extract the committed witness.
 - **Prove phase:** The prover then uses $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$ to prove an NP-statement over the previously committed values. The witness for this proof consists of the committed values along any additional randomness, as required by $\mathcal{P}_{\text{com}}$. The statement consists of the statement argument of the $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ functionality along the commitments generated by $\mathcal{P}_{\text{com}}$.

5.2 Specification of $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$

The functionality $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$ allows any process to act as a leader and initiate the generation of a common random value. Later, this functionality will be used in conjunction with the accountable one-to-many commit-and-prove functionality $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$, where a process proves compliance with a protocol with respect to a random value $r = r^{\text{com}} + r^{\text{coin}}$, where r^{com} is privately chosen and committed to $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$, and r^{coin} is the result of an independently uniformly sampled leader-based coin-toss. This ensures that the proof is both private (due to r^{com}) and unbiased (due to the entropy of r^{coin}). The ideal functionality $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$ is specified in Functionality 2, and realized in Appendix B.

5.3 Use of $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$

As in the CLOS compiler [44], our construction employs the commit-and-prove paradigm adapted to the accountable setting. Specifically, $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ is used to enforce honest behavior by requiring parties to prove statements about some (genuine) behavior fragments, playing the role of (secret) committed witnesses. The proof asserts that a party followed its prescribed protocol, using internal randomness and inputs as prescribed.

Crucially, the randomness used to generate a (genuine) behavior fragment β must be tied to the party’s committed random tape. This random tape is constructed as described in Section 5.2. As discussed in Appendix B, the functionality $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$ guarantees liveness, agreement, and unpredictability in the nominal case ($f \leq t_\epsilon$), and provides accountability in the degraded case ($f > t_\epsilon$), where inconsistencies expose a significant set of faulty processes. Importantly, the random tape must be sampled *once*, during an (asynchronous) *offline setup phase*, independently of the specific protocol to be emulated during the online phase.

In this context, the functionality $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ will be used for the following relationship:

Functionality 2 $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$ **Variables:**

$\text{Map}([\text{SID} \times \text{Process}] \rightarrow \text{Set}(\text{Process})) \ I \leftarrow \emptyset$: Initiators of the coin-tossing
 $\text{Map}([\text{SID} \times \text{Process}] \rightarrow \text{Rand}) \ r \leftarrow \emptyset$: the common coin to be returned

Events:

request (toss_req, sid, L): Requesting a common coin associated to a leader L

notification (toss_res, sid, L, r): Receiving a common coin associated with leader L

Functionality:

Upon receiving (toss_req, sid, L) from $p_j \in \Psi$:

if $r[\text{sid}, L] = \perp$: $r[\text{sid}, L] \xleftarrow{\$} \text{Rand}$

► Randomness/Unpredictability

$I[\text{sid}, L] \leftarrow I[\text{sid}, L] \cup \{p_j\}$, and notify Sim with (toss, sid, L, p_j)

Upon $|I[\text{sid}, L] \cap \text{Correct}| = n - t \wedge L \in I[\text{sid}, L]$:

► Obligation

send (toss_res, sid, L, $r[\text{sid}, L]$) to Sim and each $p_j \in \Psi$ as a delayed output

Upon $|I[\text{sid}, L]| = n - t_\epsilon \wedge L \in I[\text{sid}, L]$ and requested by Sim:

► Totality

send (toss_res, sid, L, $r[\text{sid}, L]$) to Sim and each $p_j \in \Psi$ as a delayed output

// degraded mode ($t_\epsilon < f$)

Upon receiving (BreakLiveness) from Sim, Sim can indefinitely delay any output (except detection).

Upon receiving (BreakAgreement, sid, L, π) from Sim, if $\forall i \in [n]$ the judge, on input $\pi[i]$, returns the (fair) verdict $\text{dis}(D_i)$ with $|D_i| \geq d$, then:

allow two correct processes to deliver (toss_res, sid, L, r) and (toss_res, sid, L, $r' \neq r$).

► Disagreement

if $f \leq t_{\text{acc}}$: send (detection, sid, L, D_i , $\pi[i]$) to each p_i as a delayed output.

► Accountability

// critical mode ($n - t_\epsilon \leq f$)

Randomness is not guaranteed anymore, i.e. for any process L, Sim can choose any value r as payload for a notification of the form (toss_res, sid, L, r).

$$\begin{aligned} \mathcal{R}_{\text{r}_{\text{coin}}[]} = \{ & (\check{\beta}_k, [r_{\text{com}}, \beta_1, \dots, \beta_k]) \mid \forall j \in [1, k], \beta_j \in \text{HTFrag} \wedge \\ & \forall j \in [2, k], \beta_j.\text{first_state} = \beta_{j-1}.\text{last_state} \wedge \\ & \text{trace}(\beta_k) = \check{\beta}_k \wedge \\ & \beta.\text{rand} = r_{\text{com}} + \text{r}_{\text{coin}}[p] \text{ with} \\ & \beta \triangleq (\beta_1 \frown \dots \frown \beta_k) \wedge p \triangleq \beta.\text{process} \} \end{aligned}$$

Here, the statement is a trace fragment, where a trace fragment is the projection over received/sent messages of some genuine behavior fragment, which plays the role of the witness (along with already committed behavior fragments, and committed randomness). The randomness used during the computation of the entire behavior $\beta = (\beta_1 \frown \dots \frown \beta_k)$ is denoted $\beta.\text{rand}$, and is supposed to be the sum of the randomness r_{com} committed during the very first commitment of $p = \beta.\text{process}$, and the common coin $\text{r}_{\text{coin}}[p]$ subsequently obtained from $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$, under the leadership of p . Details on the meaning of $\mathcal{R}_{\text{r}_{\text{coin}}[]}$ are given in the next subsection, but can be reasonably skipped during the first read.

5.4 Abstract Machines for Commit&Prove

To discuss more precisely, the NP-statements proven via $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$, we have to give a more formal description of the involved machines. We adopt the classical approach of encapsulating computational restrictions within abstract state machines, whose operations can be represented by collections of Turing machines with bounded runtime [13, 14, 16, 24, 38–41, 100, 108, 109]. In this context, we focus on the main parties in our message-passing protocols. These parties interact with the ideal authenticated network $\mathcal{F}_{\text{auth}}$, receive inputs and send outputs from/to the environment (which plays

the role of an arbitrary application layer), and interact with ideal cryptographic functionalities. Without loss of generality, when a process in some state s receives an input ι from the environment or a message from the network, it executes a (polynomially bounded) sequence of steps. These steps may involve interactions with cryptographic ideal functionalities and local randomness r . The process concludes its activation in a new state s' , while preparing a (potentially $\perp$) output o for the environment and a (possibly empty) set of messages to send to the network.

Abstract machine. More formally, let $\mathcal{M}$ denote the set of messages. Each message $m \in \mathcal{M}$ encodes (1) its sender, denoted by $m.\text{sender} \in \Psi$, and (2) its recipient, denoted by $m.\text{receiver} \in \Psi$. Let $\mathcal{S}$ denote the set of states. Each state $s \in \mathcal{S}$ encodes the process associated with s (denoted by $s.\text{process} \in \Psi$). For each process $p_i \in \Psi$, there exists a unique state $s_i^0 \in \mathcal{S}$, such that $s_i^0.\text{process} = p_i$ deemed as an initial state. Protocol Π_r (parametrized by random tape $r \in \text{Rand} \triangleq \{0, 1\}^{\kappa_r}$, with $\kappa_r \leq \kappa$)³ maps (1) the state of a process at the start of a transition, and (2) the input notification (either the reception $\text{rcv}(m_i) \in \mathcal{I}_{\text{msg}}$ of a message $m_i \in \mathcal{M}$ or a new input $\iota \in \mathcal{I}_{\text{app}}$ from the reactive application layer) that triggered the transition into (a) a new state of the process, and (b) the message the process sends in the next round. Formally, given (1) a state $s \in \mathcal{S}$, and (2) an input notification $\iota \in \mathcal{I} \triangleq \mathcal{I}_{\text{msg}} \uplus \mathcal{I}_{\text{app}}$, $\Pi_r(s, \iota) = (s', M, o)$, where $M \subset \mathcal{M}$ is the next (potentially empty) set of messages to be sent, while $\forall (m_i, m_j) \in M^2, m_i \neq m_j \implies m_i.\text{receiver} \neq m_j.\text{receiver}$, $s' \in \mathcal{S}$ is a state such that $s'.\text{process} = s.\text{process}$, and $o \in \mathcal{O}$ is an output notification (by default, $o = \perp$). We also consider a separate set $\mathcal{I}_{\text{adv}}$ of input notification, that includes the special corruption notifications $\{\text{corrupt}(p_j) | p_j \in \Psi\}$. An *event* is a tuple $e = (r, s, \iota, s', M, o) \in \text{Rand} \times \mathcal{S} \times (\mathcal{I} \cup \mathcal{I}_{\text{adv}}) \times \mathcal{S} \times 2^{\mathcal{M}} \times \mathcal{O}$. We note $e.\text{rand} = r$, $e.\text{old_state} = s$, $e.\text{input} = \iota$, $e.\text{new_state} = s'$, $e.\text{sentMsgs} = M$, and $e.\text{output} = o$. A *genuine event* is an event e such that $e.\text{input} \in \mathcal{I}$ and $\Pi_r(s, \iota) = (s', m', o)$. A corrupted (resp., corruption) event is an event e such that $e.\text{input} \in \mathcal{I}_{\text{adv}}$ (resp., $e.\text{input} = \text{corrupt}(p_j)$ with $p_j = e.\text{old_state}.\text{process}$).

Behaviors. A sequence of events $\beta = e_1, e_2, \dots, e_k, e_{k+1}, \dots$ is a behavior fragment of a process p_i associated to random tape r if and only if:

- (1) (Fixed process) $e_1.\text{old_state}.\text{process} = p_i$
- (2) (Fixed random tape) $\forall j \in [1, |\beta|], e_j.\text{rand} = r$.
- (3) (Transition consistency) $\forall j \in [1, |\beta| - 1], e_{j+1}.\text{old_state} = e_j.\text{new_state}$
- (4) (Corruption consistency) If any, the first corrupted event is a corruption event

In such a case, we note $\beta.\text{rand} = r$, and $\beta.\text{process} = p_i$. A behavior fragment is said *genuine* if it is composed by genuine events only. We note $\beta \frown \beta'$ the concatenation of two behavior fragments. Let $m \in \mathcal{M}$ and e appearing in β . If $e.\text{input} = \text{receive}(m)$, we say that m is received in β and e . Similarly, if $e.\text{message} = M \neq \emptyset$, we say that each $m' \in M$ is sent in β and e . If m_i and m_j are received (resp., sent) in e_i and e_j , respectively, with $i < j$, we say that m_i is received (resp., sent) before m_j in β . If no messages are sent in β , we say that β is *sent-free*. A genuine behavior fragment β' of the form $\beta \frown e$ with β being sent-free, but not β' is called a *head-tail fragment*. We note HTFrgs the set of head-tail fragments. We call $\beta \triangleq \beta'.\text{tail}$ the tail of β' , and $e_\ell \triangleq \beta'.\text{head}$ the head of β' . If $\beta = e_1, e_2, \dots, e_k, e_{k+1}, \dots$ is a behavior fragment starting with an event e_1 such that $e_1.\text{old_state}$ is an initial state, we say that β is a *behavior*. Intuitively, a process's behavior describes the states and sets of sent and received messages of that process. Let $e = (r, s, \iota, s', M, o)$ be an event. The *trace* of e , noted $\text{trace}(e)$, is the pair $(\{\iota\} \cap \mathcal{I}_{\text{msg}}, M)$, i.e. the projection over received and sent messages. Let $\beta = e_1, \dots, e_\ell \in \text{HTFrgs}$. The *trace* of β , noted $\text{trace}(\beta)$, is defined as $\text{trace}(e_1), \dots, \text{trace}(e_\ell)$.

³For simplicity [44], we assume that r is sufficiently long for any execution of a protocol Π ; a pseudorandom generator can be used to ensure this for any Π [21].

In summary, the trace $\tilde{\beta}$ of a head-tail fragment β represents the order-preserving list of plaintext received and sent messages.

5.5 The Compiler τ_{zk-scr}

We are finally ready to present the compiler τ_{zk-scr} . Essentially, τ_{zk-scr} works as τ_{scr} where the functionality $\mathcal{F}_{rbc}^{acc}$ has been “replaced by” $\mathcal{F}_{cp}^{1:M,acc}$.

Intuition. Consider a distributed system Ψ with $|\Psi| = n$ processes executing a protocol $\mathcal{P}$. Now imagine an idealized oracle θ integrated into the system with the following functionalities:

- (1) *Message relaying*: All communication between processes is routed through θ . Specifically, when a process $p \in \Psi$ intends to send messages $m_1, \dots, m_k$ to $q_1, \dots, q_k \in \Psi$ respectively, p sends $M = \{m_1, \dots, m_k\}$ to θ which then forwards $m_1, \dots, m_k$ to $q_1, \dots, q_k$, respectively, following a first-in-first-out (FIFO) policy.
- (2) *Soundness verification*: Whenever a process sends a set of messages M to θ (to have the messages relayed to its recipient), the process accompanies the message with the corresponding head-tail behavior fragment $\beta_{meantime}$. This allows θ to verify the process’s soundness before relaying any message. In the meantime, θ maintains a (genuine) behavior $current_p$, for each process $p \in \Psi$, initialized to an initial state. The oracle θ performs the following checks upon receiving $(M, \beta_{meantime})$ from process p :
 - (a) It verifies that $\beta_{meantime}$ is valid.
 - (b) It checks that the first state of $\beta_{meantime}$ matches the final state of $current_p$.
 - (c) It ensures that all incoming messages referenced in $\beta_{meantime}$ were previously relayed by θ in the specified order.

If these checks pass:

- (a) $current_p$ is updated to $current_p \widehat{\beta_{meantime}}$.
- (b) The messages in M are forwarded to their respective recipients.

Otherwise, θ permanently excludes p from the system, ignoring all future messages from p , including m .

Due to the described construction, θ observes an execution that is effectively *benign*, meaning all processes are either correct or behave as if they have crashed. The Bracha simulation achieves this by emulating θ on top of the reliable broadcast functionality $\mathcal{F}_{rbc}$. The transformation τ_{scr} builds on this idea but replaces $\mathcal{F}_{rbc}$ with its accountable counterpart, $\mathcal{F}_{rbc}^{acc}$. The design of τ_{scr} consists of the following components:

- (1) *State-Machine Layer*: This layer governs the process’s behavior in accordance with the original protocol $\mathcal{P}$. It dictates which messages should be sent and when, mirroring the intended logic of the protocol.
- (2) *Secure Broadcast Layer*: Messages generated by the state-machine layer are transmitted through the secure broadcast layer. In the nominal mode, this layer ensures that:
 - (a) All processes secure-deliver the same set of messages.
 - (b) Messages from the same sender are secure-delivered in the order they were secure-broadcast.

These guarantees are achieved using $\mathcal{F}_{rbc}$. When $\mathcal{F}_{rbc}$ is replaced with $\mathcal{F}_{rbc}^{acc}$, any equivocation in the degraded mode results in exposing malicious processes, thereby ensuring accountability.

- (3) *Soundness Layer*: This layer simulates the correctness verification duties of θ by validating the secure-delivered messages to maintain a benign execution of the system. Unlike the state-machine layer, which operates on a single process (the “host”), the soundness layer addresses the entire system and operates locally, irrespective of the number of faulty processes.

A challenge arises when the soundness layer requires access to private information. For example, if messages are encrypted to preserve privacy, verifying the recipient's behavior may necessitate decrypting the message to feed it into the simulated state-machine. Similarly, randomness involved in the protocol must remain unrevealed to preserve its unpredictability, which is essential for its purpose. To address these concerns, τ_{zk-scr} incorporates $\mathcal{F}_{cp}^{1:M,acc}$ instead of $\mathcal{F}_{rbc}^{acc}$, enabling a privacy-preserving soundness layer. This ensures that private inputs, such as encrypted messages or secret randomness, remain private while still allowing the soundness layer to function effectively. The main modifications due to this substitution are colored in **green** in Algorithm 1.

In our construction, every statement proven via the functionality $\mathcal{F}_{cp}^{1:M,acc}$ refers to a random tape r formed as the sum of two independent contributions: a *private* component r_{com} , and a *public* component r_{coin} . The private contribution r_{com} is committed via $\mathcal{F}_{cp}^{1:M,acc}$ at line 11, while the public contribution r_{coin} is obtained through the accountable one-to-many leader-based coin-tossing functionality $\mathcal{F}_{lb-coin}^{1:M,acc}$ at line 13. The resulting random tape $r = r_{com} + r_{coin}$ is used to instantiate the randomness of the simulated protocol Π within the compiled execution. This split guarantees that as long as one of the two contributions is independently and uniformly sampled (typically r_{coin}), then the joint value r remains unbiased. Furthermore, because r_{com} is privately committed, its value remains hidden from the verifier(s) during the proof, ensuring the zero-knowledge property of the witnesses relative to the statements. Passages relative to the random tape generation are colored in **red** in Algorithm 1.

Pseudocode. The pseudocode encapsulates the implementation of the various modules, focusing on the sequence of actions triggered when a process is instructed to send or receives a message. Specifically, it defines the interplay between the state-machine layer and the soundness module.

In Algorithm 1, every time the state-machine layer receives (line 16) an input ι , either from the application layer ($\iota \in \mathcal{I}_{app}$) or the soundness layer ($\iota \in \mathcal{I}_{msg}$), it computes its next state (line 17) and appends the corresponding event to its current head-tail behavior fragment stored in memory (line 18). When the state-machine intends to send messages (line 21), the soundness module takes the following steps (after having incremented its sequence number idx line 21): (1) it commits to its current head-tail behavior fragment $\beta_{meantime}$ (line 23), and (2) sends (line 24) the associated statement trace($\beta_{meantime}$).

When a process receives an output notification from $\mathcal{F}_{cp}^{1:M,acc}$ (line 26), it does not immediately pass the associated messages to the state-machine layer, even if a message is intended for the process. Instead, the process stores the behavior trace in the *Traces* data structure (line 27), which tracks all verified behavior traces. Messages are only propagated to the state-machine layer when they are part of a validated execution trace, i.e., when they belong to the *validated* set.

A message becomes validated under the following conditions: (1) all previously sent messages from the same sender are validated and (2) all received messages that justify the message are validated (line 30).

Finally, if $\mathcal{F}_{cp}^{1:M,acc}$ delivers two different behavior fragment traces associated with the same sub-session ID, it guarantees that all correct processes eventually expose the malicious processes responsible for the inconsistency.

THEOREM 1.1. *Let $\mathcal{P}$ be a distributed protocol. Let $\tilde{\mathcal{P}} = \tau_{zk-scr}(\mathcal{P})$ denote the compiled protocol, operating in the $(\mathcal{F}_{cp}^{1:M,acc}, \mathcal{F}_{lb-coin}^{1:M,acc})$ -hybrid model (which includes $\mathcal{J}$). For every active adversary $\mathcal{A}$ of $\tilde{\mathcal{P}}$, there exists a semi-honest crash-failure adversary/simulator $\mathcal{S}$ for $(\mathcal{P} \mid \mathcal{J})$, such that:*

- **Nominal mode** As long as $f \leq t_e$, $(\mathcal{A}|\tilde{\mathcal{P}})$ and $(\mathcal{S}|\mathcal{P}|\mathcal{J})$ are indistinguishable, i.e. $\tilde{\mathcal{P}}$ maintains all the hyperproperties of $\mathcal{P}$.
- **Degraded mode.** If $f > t_e$, then either:

Algorithm 1 τ_{zk-scr} : Pseudocode (for process p_i and session ID sid)

```

1: Uses:  $\mathcal{F}_{cp}^{1:M,acc}$ ,  $\mathcal{F}_{lb-coin}^{1:M,acc}$  ▷ See Functionality 1 and Functionality 2
2: Local variables:
3:  $idx_i \leftarrow 0$  ▷ Sequence number for outgoing head-tail behavior fragments
4:  $\text{Set}(\text{Messages}) \text{ validated}_i \leftarrow \emptyset$  ▷ Set of validated messages
5:  $\text{Map}(\text{Process} \times \text{Integer} \rightarrow \text{Messages}) \text{ Traces}_i[\cdot][\cdot] \leftarrow \perp$  ▷ verified behavior traces
6:  $\text{HTFragments } \beta_{\text{meantime}} \leftarrow \text{empty sequence}$  ▷ behavior since the last message sent
7:  $\text{Map}(\text{Process} \rightarrow \text{Integer}) \text{ lastVal}_i[\cdot] \leftarrow 0$  ▷ idx of last validated head-tail fragment
8: State  $s_i^{old}$ , initialized to the initial state of  $\Pi$  for  $p_i$  ▷ current simulated  $\Pi$ 's state
9:  $\text{Map}(\text{Process} \rightarrow \text{Rand}) \text{ coin}_i[\cdot]$  ▷ public randomness of processes
10:  $\text{Rand } r_i, r_i^{com}$ .  $r_i \leftarrow \perp$  and  $r_i^{com} \xleftarrow{\$} \text{Rand}$  ▷ private randomness
    // Random-Tape Generation
11: upon initialization:  $\text{send}(\text{commit}, sid, 0, r_i^{com})$  to  $\mathcal{F}_{cp}^{1:M,acc}$ 
12: upon receiving ( $\text{committed}, sid, 0, p_j$ ) from  $\mathcal{F}_{cp}^{1:M,acc}$ :
13:    $\text{send}(\text{toss\_req}, sid, p_j)$  to  $\mathcal{F}_{lb-coin}^{1:M,acc}$ 
14: upon receiving ( $\text{toss\_res}, sid, p_j, r_j^{coin}$ ) from  $\mathcal{F}_{lb-coin}^{1:M,acc}$ :
15:    $\text{coin}_i[p_j] \leftarrow r_j^{coin}$ , and if  $p_i = p_j$ :  $r_i \leftarrow r_i^{com} + r_i^{coin}$ 
    // Simulated State-Machine Layer
16: upon input\_notification ( $\iota \in \mathcal{I}_{app} \uplus \mathcal{I}_{msg}$ ): ▷  $\text{rcv}(m) \in \mathcal{I}_{msg}$  via line 33 only
17:    $(s_i^{new}, M, o) := \Pi_{r_i}(s_i^{old}, \iota)$  ▷ Wait until  $r_i \neq \perp$ 
18:    $\beta_{\text{meantime}} \leftarrow \beta_{\text{meantime}} \frown (r_i, s_i^{old}, \iota, s_i^{new}, M, o)$  ▷ keeping track of the behavior
19:    $s_i^{old} := s_i^{new}$ 
20:   if  $o \neq \perp$ : trigger  $\text{application\_output\_notification}(o)$ 
21:   if  $M \neq \emptyset$ :  $idx_i \leftarrow idx_i + 1$ , and trigger  $\text{empty\_output\_buffer}()$ 
    // Soundness Layer
22: upon empty\_output\_buffer():
23:    $\text{send}(\text{commit}, sid, idx_i, \beta_{\text{meantime}})$  to  $\mathcal{F}_{cp}^{1:M,acc}(\mathcal{R}_{\text{coin}_i[p_i]})$ 
24:    $\text{send}(\text{prove}, sid, idx_i + 1, \text{trace}(\beta_{\text{meantime}}))$  to  $\mathcal{F}_{cp}^{1:M,acc}(\mathcal{R}_{\text{coin}_i[p_i]})$ 
25:    $\beta_{\text{meantime}} \leftarrow \text{empty sequence}$ 
26: upon receiving ( $\text{verified}, sid, idx_j, p_j, \check{\beta}_j^{idx}$ ) from  $\mathcal{F}_{cp}^{1:M,acc}(\mathcal{R}_{\text{coin}_i[p_j]})$ :
27:    $\text{Traces}_i[p_j][idx_j] \leftarrow \check{\beta}_j^{idx}$ , and trigger  $\text{checkValidity}(p_j)$ 
28: upon checkValidity ( $p_j$ ):
29:    $(\check{\beta}_j, \check{\beta}_j') \leftarrow (\text{Traces}_i[p_j][\text{lastVal}_i[p_j]], \text{Traces}_i[p_j][\text{lastVal}_i[p_j] + 1])$ 
30:   if  $\check{\beta}_j' \neq \perp \wedge \check{\beta}_j'.\text{tail.rcvdMsgs} \subset \text{validated}_i$ 
      $\wedge ((\check{\beta}_j = \perp) \vee (\check{\beta}_j.\text{head.sentMsgs} \subset \text{validated}_i))$ :
31:     for each  $m \in \check{\beta}_j'.\text{head.sentMsgs}$ :
32:        $\text{validated}_i \leftarrow \text{validated}_i \cup \{m\}$ 
33:       if  $m.\text{receiver} == p_i$ : trigger  $\text{input\_notification}(\text{rcv}(m))$ 
34:        $\text{lastVal}_i[p_j] \leftarrow \text{lastVal}_i[p_j] + 1$  and trigger  $\text{checkValidity}(p_j)$ 
35:     for each  $m \in \check{\beta}_j'.\text{head.sentMsgs}$ : trigger  $\text{checkValidity}(m.\text{receiver})$ 
    // Accountability
36: upon receiving ( $\text{detection}, *, G, \pi$ ) from  $\mathcal{F}_{cp}^{1:M,acc}(*)$  or  $\mathcal{F}_{ct}^{1:M,acc}$ :
37:   trigger ( $\text{detection}, sid, G, \pi$ )

```

- (1) $\bar{\mathcal{P}}$ preserves all hypersafety properties of $\mathcal{P}$ (liveness need not hold), or
- (2) All correct processes eventually obtain externally verifiable proofs of misbehavior, exposing at least d dishonest parties, where d is the accountability parameter of $(\mathcal{F}_{cp}^{1:M,acc}, \mathcal{F}_{lb-coin}^{1:M,acc})$.

- **Complexity preservation.** Let rc and mc denote the round and message complexities of the realization of $\mathcal{F}_{cp}^{1:M,acc}$. Each message of $\mathcal{P}$ induces mc messages in $\bar{\mathcal{P}}$, and, in the nominal mode, event-related round complexities are multiplied by a factor rc .

PROOF. Foreword.

We consider the original protocol

$$\mathcal{P} = p_1 \mid \dots \mid p_n \mid \mathcal{F}_{auth}$$

and its compiled version

$$\bar{\mathcal{P}} = \bar{p}_1 \mid \dots \mid \bar{p}_n \mid \mathcal{F}_{cp}^{1:M,acc} \mid \mathcal{F}_{lb-coin}^{1:M,acc}.$$

Each process p_i implements an abstract machine Π_i . The compiled process $\bar{p}_i$ consists of three layers:

$$\bar{p}_i = \check{p}_i \mid \text{rndGen}_i \mid \text{valMng}_i,$$

where rndGen_i is the random tape generator, valMng_i is the soundness layer enforcing consistency of behavior fragments received from other processes, and $\check{p}_i$ corresponds to the state-machine layer of Π_i (instrumented with randomness obtained from rndGen_i).

Let $\mathcal{A}$ be the adversary of $\bar{\mathcal{P}}$. The simulator $\mathcal{S}$ internally simulates the adversary, denoted by $\mathcal{A}^{\text{sim}}$, and the $\mathcal{F}_{lb-coin}^{1:M,acc}$ functionality, denoted by $\mathcal{F}_{lb-coin}^{1:M,acc, \text{sim}}$. Additionally, for each $k \in [1 : n]$, $\mathcal{S}$ simulates the k -th soundness module (validation manager), denoted $\text{valMng}_k^{\text{sim}}$, the k -th random tape generator module rndGen_k , denoted $\text{rndGen}_k^{\text{sim}}$, and, upon the corruption of process p_k , it simulates a copy of the corresponding process of the compiled protocol, denoted $\bar{p}_k^{\text{sim}}$.

Messages received by $\mathcal{S}$ from the environment $\mathcal{E}$ are forwarded to $\mathcal{A}^{\text{sim}}$. Similarly, any messages sent by $\mathcal{A}^{\text{sim}}$ that are destined for the environment are forwarded to $\mathcal{E}$. The simulator $\mathcal{S}$ keeps the corruption status synchronized; that is, whenever $\mathcal{A}^{\text{sim}}$ corrupts a process $\bar{p}_k^{\text{sim}}$, the simulator also corrupts the corresponding process p_k , learns its state—including its cryptographic material and its random tape r_{p_k} . Since the simulator observes all messages handled by $\mathcal{F}_{auth}$, it can reconstruct the benign behavior β_{p_k} of process p_k up to that point, and thereafter it maintains a record of p_k 's behavior, denoted β_{p_k} as well.

Incoming messages from a corrupted process p_k to $\mathcal{S}$ are forwarded to $\mathcal{A}^{\text{sim}}$ on behalf of the corresponding corrupted process $\bar{p}_k^{\text{sim}}$. Conversely, whenever a corrupted process $\bar{p}_k^{\text{sim}}$ attempts to output a message to the environment, $\mathcal{S}$ instructs the corresponding corrupted process p_k to output the same message to $\mathcal{E}$.

Random-tape generation. First, we describe the interaction between modules $\mathcal{F}_{cp}^{1:M,acc}$, $\mathcal{F}_{lb-coin}^{1:M,acc}$, $\{\text{rndGen}_j\}_{p_j \in \Psi}$, and $\mathcal{A}$. Each compiled process $\bar{p}_j$, through rndGen_j , commits to a private random contribution r_j^{com} via a call to $\mathcal{F}_{cp}^{1:M,acc}$ (line 11), and requests a public random contribution r_j^{coin} from $\mathcal{F}_{lb-coin}^{1:M,acc}$ (line 13). The final randomness used for the random-tape of process $\bar{p}_j$ witness generation is the sum $r_j = r_j^{\text{com}} + r_j^{\text{coin}}$, computed line 15. Since r_j^{coin} is uniformly and independently (from r_j^{com}) sampled, r_j is uniformly sampled too, regardless the corruption status of $\bar{p}_j$.

The simulator emulates the interaction between $\mathcal{A}$, $\mathcal{F}_{lb-coin}^{1:M,acc}$, and each rndGen_k through their respective simulated counterparts. When $\mathcal{A}^{\text{sim}}$ instructs $\mathcal{F}_{cp}^{1:M,acc}$ to deliver a (delayed) message (committed, sid, 0, p_*) to a correct process $\bar{p}_k$, the simulator forwards it to $\text{rndGen}_k^{\text{sim}}$, mimicking the expected behavior of $\bar{p}_k$. Symmetrically, the simulator sends a message (commit, sid, 0, $\bar{p}_j$) on behalf of $\mathcal{F}_{cp}^{1:M,acc}$ to the appropriate input tape of $\mathcal{A}^{\text{sim}}$ in two situations: (1) when $\mathcal{A}^{\text{sim}}$ commands a corrupted process p_j to submit (commit, sid, 0, r_j^{com}) to $\mathcal{F}_{cp}^{1:M,acc}$, and (2) when $\mathcal{S}$ observes via $\mathcal{F}_{auth}$ that a correct process p_j has sent its first message—thus necessarily having fixed its random

tape. In both cases, the simulator updates the variable r_j^{com} in $\text{rndGen}_j^{\text{sim}}$, noted $r_j^{com, \text{sim}}$ accordingly: to the value chosen by the adversary in case (1), and to a uniformly random value in case (2).

We observe that when $\mathcal{F}_{\text{lb-coin}}^{1:\text{M}, \text{acc}, \text{sim}}$ outputs $(\text{toss_res}, \text{sid}, \bar{p}_j, r_j^{\text{coin}, \text{sim}})$, the value $r_j^{\text{sim}} \triangleq r_j^{com, \text{sim}} + r_j^{\text{coin}, \text{sim}}$ defines a uniformly random tape, consistent with the distribution of the random tape of an honest-but-curious process p_j (even if corrupted). In this case, the simulator updates its public randomness map by setting $\text{coin}[p_j] := r_j^{\text{coin}, \text{sim}}$.

Importantly, if $\mathcal{S}$ later corrupts p_j after having already sampled $r_j^{com, \text{sim}}$ uniformly at random, it can still reconstruct p_j 's random tape as r_j^{sim} without any inconsistency. The purpose of maintaining the intermediate variable $r_j^{com, \text{sim}}$ within $\text{rndGen}_j^{\text{sim}}$ is to support the sub-simulation of the interactions between $\mathcal{A}$, $\mathcal{F}_{\text{lb-coin}}^{1:\text{M}, \text{acc}}$, and each rndGen_k .

Thus, the simulator ensures that the compiled protocol's random tape of each party is consistent with the one used in the ideal execution.

Simulating protocol execution.

- **(A) Dealing with messages sent by corrupted parties:** Whenever $\mathcal{A}^{\text{sim}}$ instructs a corrupted process p_k to send

$$(\text{commit}, \text{sid}, \text{idx}, \beta_k^{\text{idx}})$$

to $\mathcal{F}_{\text{cp}}^{1:\text{M}, \text{acc}}$ for the first time at index idx , the simulator stores β_k^{idx} in the array $\underline{w}[\text{sid}, \bar{p}_k]$, as would be done internally by $\mathcal{F}_{\text{cp}}^{1:\text{M}, \text{acc}}$. The first entry of this array is reserved for the simulated committed randomness, set to

$$r_k^{com, \text{sim}} \triangleq r_k^{\text{sim}} - r_k^{\text{coin}, \text{sim}}.$$

Subsequently, when $\mathcal{A}^{\text{sim}}$ instructs $\bar{p}_k$ to send

$$(\text{prove}, \text{sid}, \text{idx}, \check{\beta}_k^{\text{idx}})$$

to $\mathcal{F}_{\text{cp}}^{1:\text{M}, \text{acc}}$, and the pair

$$(\check{\beta}_k^{\text{idx}}, \underline{w}[\text{sid}, \bar{p}_k]) \in \mathcal{R}_{\text{coin}[\cdot]},$$

the simulator instructs the underlying process p_k to send

$$(\text{send}, \text{sid}, M = (m_a, \dots, m_z), (p_a, \dots, p_z))$$

to $\mathcal{F}_{\text{auth}}$, where each m_q in M is intended for recipient p_q , and M is defined as

$$M \triangleq \underline{w}[\text{sid}, \bar{p}_k][\text{idx}].\text{sentMsgs}.$$

- **(B) Dealing with genuine messages:** The simulator $\mathcal{S}$ records all messages sent, received, or leaked through $\mathcal{F}_{\text{auth}}$, maintaining a complete trace for each process. Upon observing the idx -th send request issued by process p_i , the simulator generates the corresponding genuine trace fragment $\check{\beta}_i^{\text{idx}}$ and writes the notification

$$(\text{verified}, \text{sid}, \text{idx}, \bar{p}_i, \check{\beta}_i^{\text{idx}})$$

on the leak tape of $\mathcal{A}^{\text{sim}}$ associated with $\mathcal{F}_{\text{cp}}^{1:\text{M}, \text{acc}}$.

- **(C) Simulating the validation procedure:** Whenever $\mathcal{A}^{\text{sim}}$ instructs that a notification

$$(\text{verified}, \text{sid}, \text{idx} > 0, \bar{p}_i, \check{\beta}_i^{\text{idx}})$$

be forwarded from $\mathcal{F}_{\text{cp}}^{1:\text{M}, \text{acc}}$ to process $\bar{p}_j$, the simulator writes the same notification to the corresponding input tape of $\text{valMng}_j^{\text{sim}}$.

Consequently, $\text{valMng}_j^{\text{sim}}$ updates its internal state as in line 27 and line 27, e.g.,

$$\text{Traces}_j[\bar{p}_i][idx] \leftarrow \check{\beta}_i^{idx}.$$

This is the only case in which $\text{valMng}_j^{\text{sim}}$ updates its state.

- **(D) Delivering the validated messages:** Whenever $\text{valMng}_j^{\text{sim}}$ triggers (line 33) the reception of a message

$$m \in \check{\beta}_i^{idx}.\text{sentMsgs} \quad \text{with} \quad \check{\beta}_i^{idx} = \text{valMng}_j^{\text{sim}}.\text{Traces}_j[\bar{p}_i][idx],$$

the simulator $\mathcal{S}$ instructs $\mathcal{F}_{\text{auth}}$ to send

$$(\text{deliver}, \text{sid}, p_i, m)$$

to process p_j .

This is the only case in which such a delivery notification is issued. Note that this is sound because, by the construction above, a matching request

$$(\text{send}, \text{sid}, p_i, M = (\dots, p_j, \dots), (\dots, m, \dots))$$

must have been issued earlier by process p_i to $\mathcal{F}_{\text{auth}}$.

Nominal case. In this setting, the adversary corrupts at most t processes. The simulator implements a full forward simulation from the real-world execution $(\mathcal{A} \mid \bar{\mathcal{P}})$ to the ideal-world execution $(\mathcal{S} \mid \mathcal{P})$, sufficient to guarantee hypersafety preservation.

Specifically, for any pair of correct processes $(\bar{p}_i, \bar{p}_j)$ and any index (k, idx) , the corresponding values $\text{Traces}_i[\bar{p}_k][idx]$ and $\text{Traces}_j[\bar{p}_k][idx]$ remain consistent by the agreement property of $\mathcal{F}_{\text{cp}}^{1:\text{M}, \text{acc}}$. For each process $\bar{p}_k$, let $\check{\beta}_k^*$ denote the longest trace fragment consistently reconstructed across all correct $\text{valMng}_j^{\text{sim}}$, that is, extracted from the set

$$\{\text{valMng}_j^{\text{sim}}.\text{Traces}_j[\bar{p}_k]\}_{\bar{p}_j \in \text{Correct}}.$$

In this sense, all correct parties agree on a common prefix of $\check{\alpha}^* \triangleq \{\check{\beta}_k^*\}_{\bar{p}_k \in \Psi}$.

Finally, by construction of the simulator and the knowledge soundness of $\mathcal{F}_{\text{cp}}^{1:\text{M}, \text{acc}}$, we ensure that the behavior of each real process p_k aligns with its abstract trace $\check{\beta}_k^*$, thus realizing the abstract execution $\check{\alpha}^*$.

Liveness is ensured by the totality property of $\mathcal{F}_{\text{cp}}^{1:\text{M}, \text{acc}}$, which guarantees that all correct processes eventually agree on the set $\check{\alpha}^* \triangleq \{\check{\beta}_k^*\}_{\bar{p}_k \in \Psi}$. This shared view prevents any deadlock in the simulation of the validation mechanism.

Concretely, steps (A) and (B) ensure that whenever a message m is (necessarily honestly) sent via $\mathcal{F}_{\text{auth}}$, the corresponding genuine trace fragment $\check{\beta}$ is propagated to $\mathcal{A}^{\text{sim}}$ through the leak tape linked to $\mathcal{F}_{\text{cp}}^{1:\text{M}, \text{acc}}$. In a fair real-world execution, $\mathcal{A}^{\text{sim}}$ will eventually instruct this notification to be delivered to the appropriate correct processes. Then, in step (C), $\mathcal{S}$ forwards it to their respective $\text{valMng}_j^{\text{sim}}$ components, mimicking the compiled protocol's behavior.

As a result, the full trace $\check{\alpha}^*$ is eventually reconstructed across all simulated managers' trace stores (i.e., $\text{valMng}_*^{\text{sim}}.\text{Traces}_*$), enabling $\mathcal{S}$ to instruct $\mathcal{F}_{\text{auth}}$ to deliver m . Hence, the ideal execution remains fair whenever the real execution is fair.

Degraded case. In this case, the adversary can corrupt up to t_{acc} processes. The only possible safety violation is agreement. Assume that the agreement property of $\mathcal{F}_{\text{zkPoK}}^{1:\text{M}, \text{acc}}$ is violated (which implies that $f > t_\epsilon$). In this case, accountability is inherited from the accountability guarantees of $\mathcal{F}_{\text{cp}}^{1:\text{M}, \text{acc}}$ or $\mathcal{F}_{\text{lb-coin}}^{1:\text{M}, \text{acc}}$.

□

A COMMIT&PROVE: REALIZING $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ IN THE $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$ -HYBRID MODEL

In this section, we present our realization of $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ in the $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$ -hybrid model, assuming a UC-secure commitment scheme.

Before specifying the functionality $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$ in Appendix A.2, we present in Appendix A.1 the commitment primitive involved in the “commit part” of the *commit-and-prove* functionality.

A.1 Non-interactive Commitment

We assume a non-interactive commitment scheme [82], defined by a deterministic algorithm $\text{Com}(\cdot, \cdot)$. On input a witness w and a random value r , the algorithm Com generates a commitment $\hat{w} := \text{Com}(w, r)$. The usual Reveal and CheckReveal protocols are straightforward: Reveal consists of disclosing the full triplet $(\hat{w}, w, r)$. CheckReveal verifies the validity of a commitment by checking whether $\hat{w} = \text{Com}(w, r)$. The commitment scheme satisfies the following properties:

- **Binding:** It is computationally infeasible to find (w, w', r, r') with $(w, r) \neq (w', r')$ such that $\text{Com}(w, r) = \text{Com}(w', r')$.
- **Hiding:** For any two values w and w' , and uniformly random values R and R' , the commitments $\text{Com}(w, R)$ and $\text{Com}(w', R')$ are computationally indistinguishable.

An adaptively UC-secure realization of a non-interactive commitment functionality $\mathcal{F}_{\text{nicom}}$ has been presented in [43] in the (uniformly sampled) common reference string $\mathcal{F}_{\text{crs}}$ -hybrid model. A more efficient realization appears in [78].

A.2 Specification of $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$

Here, one party proves a statement to all the parties in Ψ . This means that, in the nominal mode ($f \leq t_\epsilon$), if a process is convinced by a statement, then this becomes eventually the case for all correct processes. In the degraded mode ($t_\epsilon < f \leq t_{\text{acc}}$), if two correct processes are convinced by two different statements for the same session ID, some malicious processes are eventually exposed.

The definition of the accountable one-to-many zero-knowledge functionality, denoted $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$, appears in Functionality 3. The detailed interaction with the simulator are omitted. Essentially, the simulator (1) controls the scheduling of the outputs, (2) chooses the nominal mode or the degraded mode, (3) in the degraded mode, chooses to violate liveness, or agreement (after having corrupted $f > t_\epsilon$ processes). Let us note that even in the degraded mode, the simulator cannot tackle soundness, knowledge extractability, and zero knowledge, i.e. it cannot convince of a false statement or the knowledge of some witness it does not know, or learn anything from a correct process beyond the proven statements. If a disagreement occurs, it necessarily implies that $f > t_\epsilon$, while all correct processes are guaranteed to eventually receive some externally-verifiable proof-of-misbehavior exposing d dishonest processes.

The $\mathcal{ABC}$ transformation [53, Theorem 5] can be used to realize $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$, by sequentially composing its non-accountable counterpart $\mathcal{F}_{\text{zkPoK}}^{1:\text{M}}$ with the accountable confirmer functionality $\mathcal{F}_{\text{conf}}^{\text{acc}}$, itself UC-realized in [53, Appendix B]. It is straightforward to realize $\mathcal{F}_{\text{zkPoK}}^{1:\text{M}}$, by reliable-broadcasting a UC-secure non-interactive ZKPoK (NIZKPoK) through the $\mathcal{F}_{\text{rbc}}$ functionality. An alternative realization on top of potentially interactive ZKPoK has been proposed in [91]. Both approaches are examined in Appendix C, along with presentations of the corresponding fundamental functionalities $\mathcal{F}_{\text{nizkpok}}$ and $\mathcal{F}_{\text{zkipok}}$.

A.3 Pseudocode Description of $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$'s Realization

Following [44, §7], we present a protocol $\mathcal{P}_{\text{cp}}^{1:\text{M},\text{acc}}$ that UC-realizes the accountable one-to-many commit-and-prove functionality $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ in the $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$ -hybrid model, assuming a non-interactive

Functionality 3 $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$

Parameters:

$\mathcal{R}$: An NP relation for an NP language.

d : The number of dishonest processes exposed in case of equivocation in the degraded mode.

Events:

request (prove, sid, x , w): Proving knowledge of a witness for the statement x .

notification (verification, sid, P , x): Verifying that the prover P knows a witness for x .

Functionality:

// nominal mode ($f \leq t_\epsilon$)

Upon receiving (prove, sid, x , w) from P for the first time for sid, s.t. $(x, w) \in \mathcal{R}$:

send (verification, sid, P , x) directly to Sim and to each $p_j \in \Psi$ as a delayed output.

// degraded mode ($t_\epsilon < f \leq t_{\text{acc}}$)

Upon receiving (BreakLiveness) from Sim, Sim can indefinitely delay any output (except detection).

Upon receiving (BreakAgreement, sid, P , π) from Sim. If P is corrupted, and $\forall i \in [n]$, the judge, on input $\pi[i]$, returns the (fair) verdict $\text{dis}(D_i)$ with $|D_i| \geq d$, then:

accept any sound prove request even if conflicting ones were previously accepted, with corresponding notifications of the form (verification, sid, idx , P , x) and (verification, sid, idx , P , $x' \neq x$) ► Disagreement

if $f \leq t_{\text{acc}}$: send (detection, sid, p , D_i , $\pi[i]$) to each $p_i \in \Psi$ as a delayed output. ► Accountability

commitment scheme Com. The protocol is structured in two phases—*commit* and *prove*—each involving one call to $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$ with a relation tailored to the task at hand.

Commit Phase. When a process p_i receives an input (commit, sid, idx , w) (line 13), it first samples fresh randomness r (line 14) and computes the commitment $\hat{w} = \text{Com}(w, r)$ (line 17). It stores this in its local maps tracking the witnesses, randomness, and commitments (line 15, 16, and 18). Then, it proves knowledge of the committed value via a call to $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}(R_C)$ with the instance–witness pair $(\hat{w}, (w, r))$ (line 19). The relation R_C is defined as:

$$R_C = \{(\hat{w}, (w, r)) \mid \hat{w} = \text{Com}(w, r)\}.$$

Upon receiving a verification message from $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}(R_C)$ of the form (verified, sid, idx , p_j , $\hat{w}$) (line 20), each process updates its local record of commitments and triggers (committed, sid, idx , p_j) (line 21 and 22), thereby notifying the higher layer (e.g., a coin-tossing protocol) of successful commitment.

Prove Phase. Later, when a process wishes to prove a statement x about its past commitments, it is given the input (prove, sid, idx , x) (line 23). It collects the first idx committed values $w_1, \dots, w_{\text{idx}}$, their randomnesses $r_1, \dots, r_{\text{idx}}$, and the corresponding commitments $\hat{w}_1, \dots, \hat{w}_{\text{idx}}$. It then invokes $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}(R_P)$ with the instance–witness pair:

$$((x, \underline{c}), (\underline{w}, \underline{r})),$$

where the relation R_P is defined as:

$$R_P = \{((x, \underline{c}), (\underline{w}, \underline{r})) \mid \forall i, \hat{w}_i = \text{Com}(w_i, r_i) \wedge R(x, \underline{w}) = 1\}.$$

This call appears in line 25.

If a verification message of the form (verified, sid, idx , p_j , $(x_j, \underline{c})$) is later received (line 26), then every process checks that the vector $\underline{c}$ matches its record of previously accepted commitments from p_j (line 27). If so, the corresponding output (verified, sid, idx , p_j , x_j) is triggered (line 28).

Accountability. Beyond liveness, the only possible malfunction of $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$ arises in the event of an agreement violation. In such cases, detection (received at line 29) is guaranteed by the accountability property of $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$, and the resulting detection message is relayed to the level of $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ (at line 30).

Overall, this construction closely mirrors its non-accountable predecessor (as in [44, Section 7.1]) but strengthens it with accountability.

Algorithm 2 $\mathcal{P}_{\text{cp}}^{1:\text{M},\text{acc}} \leq \mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ on top of $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$: Pseudocode (for process p_i)

```

1: Parameters:
2:    $R(\cdot, \cdot)$ : The NP statement to be proven
3:    $R_C(\cdot, \cdot) \equiv \{(c, (w, r)) \mid c = \text{Com}(w, r)\}$  ▷ commitment phase
4:    $R_P(\cdot, \cdot) \equiv \{(x, \underline{c}), (\underline{w}, \underline{r}) \mid \forall i, \underline{c}[i] = \text{Com}(\underline{w}[i], \underline{r}[i]) \wedge R(x, \underline{w}) = 1\}$  ▷ prove phase
5: Uses:
6:    $\text{Com}(\cdot, \cdot)$  ▷ Deterministic non-interactive commitment scheme
7:    $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}(R_C)$  ▷ Prove correctness of commitment
8:    $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}(R_P)$  ▷ Prove validity of statement  $x$  relative to committed witness
9: Local variables:
10:  Map(SIDs  $\rightarrow$  Array(Rand))  $\underline{r}_i$  ▷ randomness
11:  Map(SIDs  $\rightarrow$  Array(String))  $\underline{w}_i$  ▷ witness
12:  Map( $\Psi \times$  SIDs  $\rightarrow$  Array(String))  $\hat{\underline{w}}_i$  ▷ committed witnesses
13: upon input (commit, sid, idx, w):
14:    $r \xleftarrow{\$} \text{Rand}$ 
15:    $\underline{r}_i[\text{sid}].\text{append}(r)$ 
16:    $\underline{w}_i[\text{sid}].\text{append}(w)$ 
17:    $\hat{\underline{w}}_i^{\text{idx}} \leftarrow \text{Com}(w, r)$ 
18:    $\hat{\underline{w}}_i[p_i, \text{sid}].\text{append}(\hat{\underline{w}}_i^{\text{idx}})$  ▷ Commit to the witness with fresh randomness
19:   send (prove, sididx,  $\hat{\underline{w}}_i^{\text{idx}}$ , (w, r)) to  $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}(R_C)$ 
20: upon receiving (verified, sididx,  $p_j$ ,  $\hat{\underline{w}}_j^{\text{idx}}$ ) from  $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}(R_C)$ :
21:    $\hat{\underline{w}}_i[p_j, \text{sid}][\text{idx}] \leftarrow \hat{\underline{w}}_j^{\text{idx}}$ 
22:   trigger (committed, sid, idx,  $p_j$ )
23: upon input (prove, sid, idx, x):
24:   ( $\text{stmt}, \text{wit}$ )  $\leftarrow ((x, \hat{\underline{w}}_i[p_i, \text{sid}][\text{idx}]), (\underline{w}_i[\text{sid}][\text{idx}], \underline{r}_i[\text{sid}][\text{idx}]))$ 
25:   send (prove, sididx, wit, stmt) to  $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}(R_P)$ 
26: upon receiving (verified, sididx,  $p_j$ , ( $x_j$ ,  $\hat{\underline{w}}_j$ )) from  $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}(R_P)$ :
27:   if  $\hat{\underline{w}}_j = \hat{\underline{w}}_i[p_j, \text{sid}][\text{idx}]$ :
28:     trigger (verified, sid, idx,  $p_j$ ,  $x_j$ )
29: upon receiving (detection, sididx,  $G, \pi$ ) from  $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}(R_C)$  or  $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}(R_P)$ :
30:   trigger (detection, sididx,  $G, \pi$ ) ▷ Detected equivocation in degraded mode

```

THEOREM A.1. Let $\mathcal{R}$ be an NP relation. Let $\mathcal{P}_{\text{cp}}^{1:\text{M},\text{acc}}$ be the protocol described in Algorithm 2, parametrized with $\mathcal{R}$, and let $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ be the functionality described in Functionality 1, parametrized with $\mathcal{R}$. Then, $\mathcal{P}_{\text{cp}}^{1:\text{M},\text{acc}}$ UC-realizes $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ in the $(\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}})$ -hybrid model.

PROOF.

Foreword.

We consider the ideal functionality

$$\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}(R) = p_1 \mid \dots \mid p_n \mid \tilde{\mathcal{F}}_{\text{cp}}^{1:\text{M},\text{acc}}(R).$$

where $\tilde{\mathcal{F}}_{\text{cp}}^{1:\text{M},\text{acc}}(R)$ corresponds to the ideal functionality $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}(R)$ deprived of the dummy processes $p_1, \dots, p_n$, and parametrized with NP-relation R .

We work in the $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$ -hybrid model and analyse the real protocol

$$\mathcal{P}_{\text{cp}}^{1:\text{M},\text{acc}} = \bar{p}_1 \mid \dots \mid \bar{p}_n \mid \mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}(R_C) \mid \mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}(R_P),$$

whose pseudocode is given in Algorithm 2.

Let $\mathcal{A}$ be the adversary of $\mathcal{P}_{\text{cp}}^{1:\text{M},\text{acc}}$. The simulator $\mathcal{S}$ internally simulates the adversary, denoted by $\mathcal{A}^{\text{sim}}$, and for each corrupted process $\bar{p}_k \in \text{Corrupted}$, it simulates a copy, denoted $\bar{p}_k^{\text{sim}}$, and corrupts p_k .

Messages received by $\mathcal{S}$ from the environment $\mathcal{E}$ are forwarded to $\mathcal{A}^{\text{sim}}$. Similarly, any messages sent by $\mathcal{A}^{\text{sim}}$ that are destined for the environment are forwarded to $\mathcal{E}$. Incoming messages from a corrupted process p_k to $\mathcal{S}$ are forwarded to $\mathcal{A}^{\text{sim}}$ on behalf of the corresponding corrupted process $\bar{p}_k^{\text{sim}}$. Conversely, whenever a corrupted process $\bar{p}_k^{\text{sim}}$ attempts to output a message to the environment, $\mathcal{S}$ instructs the corresponding corrupted process p_k to output the same message to $\mathcal{E}$.

Simulator construction. Throughout the simulation $\mathcal{S}$ maintains the public map $\hat{w}$ —mirrors of the honest parties' local variable—indexed by the *sid* of the session under consideration.

- **(A) Honest Commitment:**

Upon receiving a notification (committed, *sid*, *idx*, p_j) from $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ for some $p_j \in \text{Correct}$, the simulator $\mathcal{S}$ samples a fresh dummy commitment $\hat{w}_j \leftarrow \text{Com}(0^\kappa, r)$ for a random r , and stores it in $\hat{w}[\text{sid}, p_j][\text{idx}]$. Then, $\mathcal{S}$ writes

$$(\text{verified}, \text{sid}_{\text{idx}}, p_j, \hat{w}_j)$$

to the leak tape of $\mathcal{A}^{\text{sim}}$ connected to $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}(R_C)$.

- **(B) Honest Proof:**

Upon receiving a notification (verified, *sid*, *idx*, p_j , x_j) from $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ for some $p_j \in \text{Correct}$, the simulator $\mathcal{S}$ looks up the previously stored dummy commitments and writes

$$(\text{verified}, \text{sid}_{\text{idx}}, p_j, (x_j, \hat{w}_j[\text{sid}, p_j][\text{idx}]))$$

to the leak tape of $\mathcal{A}^{\text{sim}}$ connected to $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}(R_P)$.

- **(C) Malicious Commitment:**

When $\bar{p}_j^{\text{sim}}$ (under instruction from $\mathcal{A}^{\text{sim}}$) sends for the first time the message

$$(\text{prove}, \text{sid}_{\text{idx}}, \hat{w}_j, (w_j, r_j)) \quad \text{to} \quad \mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}(R_C),$$

with $(\hat{w}_j, (w_j, r_j)) \in R_C$ (as per line 19), then $\mathcal{S}$ sets

$$\hat{w}[\text{sid}, p_j][\text{idx}] := \hat{w}_j,$$

and instructs the ideal-world process p_j to send (commit, *sid*, *idx*, w_j) to $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$.

Later, when $\mathcal{S}$ is notified by $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ of (committed, *sid*, *idx*, p_j) for $p_j \in \text{Corrupted}$, it retrieves the stored commitment and writes

$$(\text{verified}, \text{sid}_{\text{idx}}, p_j, \hat{w}[\text{sid}, p_j][\text{idx}])$$

to the leak tape of $\mathcal{A}^{\text{sim}}$ associated with $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}(R_C)$.

- **(D) Malicious Proof:**

Similarly, when $\bar{p}_j^{\text{sim}}$ initiates for the first time a call to

$$(\text{prove}, \text{sid}_{\text{idx}}, (x_j, \hat{w}_j), (w_j, r_j)) \quad \text{to} \quad \mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}(R_P),$$

with $((x_j, \hat{w}_j), (\underline{w}_j, \underline{r}_j)) \in R_P$ and $\hat{w}_j = \hat{w}[\text{sid}, p_j][\text{idx}]$ (as per line 25), then $\mathcal{S}$ instructs p_j to send $(\text{prove}, \text{sid}, \text{idx}, x_j)$ to $\mathcal{F}_{\text{cp}}^{1:\text{M}, \text{acc}}$.

Upon receiving a notification $(\text{verified}, \text{sid}, \text{idx}, p_j, x_j)$ from $\mathcal{F}_{\text{cp}}^{1:\text{M}, \text{acc}}$, the simulator writes

$$(\text{verified}, \text{sid}_{\text{idx}}, p_j, (x_j, \hat{w}_j[\text{sid}, p_j][\text{idx}]))$$

to the leak tape of $\mathcal{A}^{\text{sim}}$ connected to $\mathcal{F}_{\text{zkPoK}}^{1:\text{M}, \text{acc}}(R_P)$.

Indistinguishability argument. We define the *hybrid world* to be the ideal execution in which, for each honest party p_j , the dummy commitment $\hat{w}_j^{\text{idx}}$ used in the simulator is replaced by the actual commitment $\hat{w}_j^{\text{idx}} = \text{Com}(w_j, r_j)$ sent by p_j in the real execution. By the binding property of the commitment scheme, each vector $\hat{w}[\text{sid}, p_j]$ uniquely determines a corresponding witness vector, even for corrupted parties. We have thus constructed a forward simulation from the real world to the hybrid world. To argue that the hybrid world is indistinguishable from the ideal world, we observe that the only difference lies in whether honest commitments encode real inputs or dummy values. If an environment could distinguish the two worlds with non-negligible advantage, then it would in particular distinguish a commitment to a real value from a commitment to a fixed dummy value (e.g., 0^λ), thereby contradicting the hiding property of the commitment scheme. This reduction follows the same structure as the argument in [44, Section 7.1].

Message delivery and fairness. All external messages are forwarded *immediately* by $\mathcal{S}$, mirroring the scheduling chosen by $\mathcal{A}^{\text{sim}}$. Because every real-world message produces exactly one ideal-world counterpart (or an explicit detection), the joint scheduling of the two executions remains lock-step and preserves fairness.

Accountability. Detection events are triggered in both worlds under exactly the same conditions: either knowledge soundness fails (impossible) or a commitment/proof is replayed inconsistently, in which case $\mathcal{F}_{\text{cp}}^{1:\text{M}, \text{acc}}$ already specifies an explicit blame set G and proof π that $\mathcal{S}$ forwards verbatim.

Remark. Universally-composable commitment schemes are typically required to provide *simulation extractability*, meaning that via a trapdoor the simulator can recover the committed value from any valid commitment and hand it to the ideal commitment functionality, thereby ensuring binding. In our $\mathcal{F}_{\text{cp}}^{1:\text{M}, \text{acc}}$'s realization, this additional property is not needed. Binding is already guaranteed by the *knowledge extractability* of the underlying $\mathcal{F}_{\text{zkPoK}}^{1:\text{M}, \text{acc}}$ functionalities. Concretely, when the (simulated) adversary instructs a corrupted process p_j to commit, it must do so by providing a valid witness–statement pair (w, c) to $\mathcal{F}_{\text{zkPoK}}^{1:\text{M}, \text{acc}}$. The simulator (simulating the real-world adversary) can directly read this pair from the corresponding tape and then instruct the corresponding ideal-world process p_j to commit to w .

□

A.4 Adaptively Secure Commit-and-Prove

In the adaptive setting, the commitment scheme must additionally provide (polynomial) *equivocality*. This means that the simulator, given a trapdoor τ for which $s = f(\tau)$, can generate (polynomially many) commitments under the same reference string s and later open each of them to *any* value of its choice. In contrast, a real committer, who does not know the trapdoor, remains bound to a unique opening, as required by the binding property of commitment schemes. Equivocality addresses the case where an honest process first generates a commitment c to some value w (unknown to the adversary), and is later adaptively corrupted, thereby revealing w . In such a situation, the simulator must be able to retroactively explain the previously issued (dummy) commitment c as a valid commitment to w , ensuring consistency with the view given to the simulated real-world adversary.

More formally, we recall the trapdoor commitment scheme following [78]’s presentation:

Trapdoor Commitment Scheme. Let $\text{TCOM} = (\text{Gen}, \text{aCom}, \text{TCom}, \text{TCol})$ be a tuple of the following four algorithms:

- **Gen**: a PPT algorithm that takes as input the security parameter κ and outputs a pair of public and trapdoor string (s, τ) .
- **aCom**: a PPT algorithm that takes as input s and a message $w \in \{0, 1\}^{\text{poly}(\kappa)}$ to be committed, chooses $r \leftarrow \text{COIN}^{\text{com}}$, and outputs $c = \text{aCom}_s(w; r)$.
- **TCom**: a PPT algorithm that takes as input τ and outputs $(c^\diamond, r^\diamond) \leftarrow \text{TCom}_\tau(1^\kappa)$.
- **TCol**: a DPT algorithm that takes $(\tau, c^\diamond, r^\diamond, w^*)$ where $w^* \in \{0, 1\}^{\text{poly}(\kappa)}$, and outputs $r^* \in \text{COIN}^{\text{com}}$ such that $c^\diamond = \text{aCom}_s(w^*; r^*)$.

We call TCOM a *trapdoor commitment scheme* if, in addition to the traditional binding and hiding properties ensured by the pair $(\text{Gen}, \text{aCom})$, the following *Trapdoor Collision Property* holds:

For any pair of string (s, τ) , generated from $\text{Gen}(1^\kappa)$ (discussed later), and all messages $w \in \{0, 1\}^{\text{poly}(\kappa)}$, the following ensembles are statistically indistinguishable in κ :

$$\left\{ (c, w, r) \mid r \leftarrow \text{COIN}^{\text{com}}, c = \text{aCom}_s(w; r) \right\}_{\kappa \in \mathbb{N}, s \in \text{Gen}(1^\kappa), w \in \{0, 1\}^{\text{poly}(\kappa)}} \stackrel{s}{\approx} \left\{ (c, w, r) \mid (c, r^\diamond) \leftarrow \text{TCom}_\tau(1^\kappa), r = \text{TCol}_\tau(c, r^\diamond, w) \right\}_{\kappa \in \mathbb{N}, s \in \text{Gen}(1^\kappa), w \in \{0, 1\}^{\text{poly}(\kappa)}}$$

Such a trapdoor commitment scheme can be instantiated from the Pedersen Commitment Scheme:

Pedersen Trapdoor Commitment. Let

$$\text{TCOM} = (\text{Gen}, \text{Com}, \text{TCom}, \text{TCol})$$

denote the Pedersen trapdoor commitment scheme [107]. Its algorithms are defined as follows, for public parameters (q, g) , q being a prime number greater than κ , and g being a generator of the cyclic group $(\mathbb{Z}_q, \cdot, 1_{\mathbb{Z}_q})$:

- **Gen** (1^κ) : sample $\tau \leftarrow \mathbb{Z}_q$, compute $s = g^\tau$, and return (s, τ) .
- **Com** $_s(w)$: to commit to $w \in \{0, 1\}^{\text{poly}(\kappa)}$, sample $r \leftarrow \mathbb{Z}_q$, and output

$$\text{Com}_s(w, r) = g^r \cdot s^w.$$

- **TCom** $_\tau(1^\kappa)$: sample $r^\diamond \leftarrow \mathbb{Z}_q$, and output $(g^{r^\diamond}, r^\diamond)$.
- **TCol** $_\tau(r^\diamond, w^*)$: on input $w^* \in \{0, 1\}^{\text{poly}(\kappa)}$, output

$$r^* = r^\diamond - w^* \cdot \tau \pmod{q}.$$

Note that in this case

$$c^\diamond = g^{r^*} \cdot s^{w^*}.$$

Moreover, we say that TCOM is *uniform-CRS compatible* if the distribution of the string s output by Gen is statistically uniform. This property holds, for instance, in the case of the Pedersen trapdoor commitment scheme.

Adaptive Security via uniform-CRS compatible Trapdoor Commitments. Our realization of $\mathcal{F}_{\text{cp}}^{1:\text{M}, \text{acc}}$ for static adversaries can be extended to achieve adaptive security by replacing the underlying commitment scheme with a *uniform-CRS compatible* trapdoor-enabled variant. The modification is as follows.

- (1) Upon its first commitment, each process p_i initiates an instance of the accountable leader-based common-coin functionality $\mathcal{F}_{\text{lb-coin}}^{1:\text{M}, \text{acc}}$, which eventually outputs a common string $s_i \in \mathbb{Z}_q$.

- (2) Commitments are then performed using the trapdoor commitment scheme aCom_{s_i} , where aCom is any *uniform-CRS compatible* trapdoor commitment scheme (such as Pedersen's), in place of the non-trapdoor scheme Com used in the static protocol.

The simulation is extended as follows. For each process p_i , the simulator locally generates a key pair $(s_i, \tau_i) \leftarrow \text{Gen}$, and publishes the public value s_i in the name of the corresponding simulated instance of $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$. When the simulator is notified by $\mathcal{F}_{\text{cp}}^{1:\text{M},\text{acc}}$ of a commitment made by some so-far honest process p_j , it produces a *dummy commitment* $c^\diamond$ with dummy randomness $r^\diamond$ by invoking TCom_{τ_j} . The simulator then forwards $c^\diamond$ to the simulated adversary, thereby maintaining consistency with the real-world execution. Later, if process p_j is adaptively corrupted and its committed value w^* is revealed, the simulator computes

$$r^* \leftarrow \text{TCol}_{\tau_j}(c^\diamond, r^\diamond, w^*),$$

which guarantees that

$$c^\diamond = \text{aCom}_{s_j}(w^*; r^*).$$

This ensures that every dummy commitment can be *explained* consistently with the value eventually revealed, thereby enabling adaptive security.

Adaptive Security with Non-Uniform CRS. If the underlying trapdoor commitment scheme aCom is not compatible with a uniform CRS, we rely on an additional initialization step to ensure simulatable setup. Each process p_j locally generates a key pair $(s_j, \tau_j) \leftarrow \text{Gen}$ and publishes the public string s_j via an invocation of $\mathcal{F}_{\text{zkpok}}^{1:\text{M},\text{acc}}$, thereby proving knowledge of τ_j such that s_j was correctly generated. Every process p_i waits to collect $n - t$ such public strings from a quorum Q_i of size $n - t$, and reliably broadcasts Q_i through $\mathcal{F}_{\text{rbc}}^{1:\text{M},\text{acc}}$. This defines for each committer p_i a vector $\vec{s}_i$ of public strings, which is then used to parameterize its trapdoor commitment scheme as

$$\text{aCom}_{\vec{s}_i}(w; \vec{r}) := (\text{aCom}_{\vec{s}_i[1]}(w; \vec{r}[1]), \dots, \text{aCom}_{\vec{s}_i[n-t]}(w; \vec{r}[n-t])).$$

The simulation strategy is essentially the same as in the uniform-CRS case. If process p_j is corrupted at the time of generating (s_j, τ_j) , the simulator directly uses the pair provided by the adversary. Otherwise, if p_j remains uncorrupted when its publication of s_j via $\mathcal{F}_{\text{zkpok}}^{1:\text{M},\text{acc}}$ occurs, the simulator itself generates a pair $(s_j, \tau_j) \leftarrow \text{Gen}$ on behalf of p_j in the simulated execution. In both cases, the simulator consistently holds all trapdoors τ_j for the public strings that will later appear in any quorum vector $\vec{s}_i$. This guarantees equivocality, enabling the simulator to apply the same commitment-explanation strategy as in the uniform-CRS setting.

B COIN-TOSSING: REALIZING $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$ IN THE $(\mathcal{F}_{\text{rbc}}^{\text{acc}}, \mathcal{F}_{\text{haccs}})$ -HYBRID MODEL

This section describes the realization of the accountable leader-based common-coin functionality $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$, presented in Functionality 2.

Our realization follows standard asynchronous coin-tossing techniques based on secret sharing, such as the Random Secret Draw [45, 73]. The output of the coin is defined over the finite group $(\mathbb{Z}_{|\text{Rand}|}, +)$ as:

$$r = \sum_{p_j \in Q_L} r_j$$

where r_j is a random value contributed by process p_j , and Q_L is a quorum of $n - t$ processes selected by the leader L , whose identity is agreed upon using $\mathcal{F}_{\text{rbc}}^{\text{acc}}$.

If at least one r_j is uniform and independent, then the sum r is itself uniform. This requires two properties: (1) each r_j is chosen before seeing the others, and (2) reconstruction occurs only after all

contributions are fixed. We enforce these using the high-threshold asynchronous complete secret sharing functionality $\mathcal{F}_{\text{hacss}}$, presented in Functionality 4.

B.1 $\mathcal{F}_{\text{hacss}}$: High-Threshold Asynchronous Complete Secret Sharing

We rely on the specification of [4] and the construction of [71, §7], which guarantees high secrecy even during the sharing phase. Secret sharing distributes a secret among n processes such that any subset of $(\ell + 1)$ can reconstruct it, but no smaller subset learns anything. Verifiable secret sharing (VSS) adds guarantees of uniqueness and binding despite a malicious dealer. A *high-threshold* VSS uses $\ell = n - t - 1$ for secrecy, rather than the minimal t . An asynchronous *complete* secret sharing (ACSS) scheme additionally ensures that if a correct process completes the sharing phase, then all correct processes eventually do so (completeness/totality).

We model this functionality as $\mathcal{F}_{\text{hacss}}$, presented in Functionality 4.

Functionality 4 $\mathcal{F}_{\text{hacss}}$

Parameters:

$\ell = n - t - 1$: A high threshold for secrecy

► Dual-Threshold

Variables:

Secret $secret \leftarrow \perp$: A secret to be shared

Set(Process) $J \leftarrow \emptyset$: the set of processes who initiated the reconstruction

Participants:

$p_1, \dots, p_n$: Parties interacting with the functionality.

Sim: Ideal adversary (simulator).

Events:

request (share, sid, s): Sharing a secret

notification (share_ack, sid, D): Acknowledging the sharing phase

request (reconstruct, sid): Initiating the reconstruction of the secret

notification (reconstruct_ack, sid, D, s): Reconstructing the secret

Functionality:

Upon receiving (share, sid, s) from the dealer D :

$secret \leftarrow s$

send (share_ack, sid, D) to Sim

send (share_ack, sid, D) to each $p_j \in \Psi$ as a delayed output.

► Correctness (Commitment/Binding)

► Notify the simulator

► Completeness

Upon receiving (reconstruct, sid) from $p_j \in \Psi$:

if $secret \neq \perp$: $J \leftarrow J \cup \{p_j\}$

Upon $|J \cap \text{Correct}| = \ell + 1 - f$ and $secret \neq \perp$:

send (reconstruct_ack, sid, D, $secret$) to Sim

► Dual-Threshold Secrecy

Upon $|J \cap \text{Correct}| = \ell$:

send (reconstruct_ack, sid, D, $secret$) to each p_j as a delayed output

- **Termination:** If the dealer is correct, and send a share request to $\mathcal{F}_{\text{hacss}}$, all honest processes eventually receive a share_ack. Similarly, if all honest processes send reconstruct requests, they eventually receive reconstruct_ack.
- **Agreement:** All honest processes reconstruct the same value.
- **Justification:** If the dealer is honest, the reconstructed value equals the one it initially shared.
- **Secrecy:** No information about the secret leaks before $\ell + 1 - f$ honest processes initiate the reconstruction.
- **Binding:** Once any honest process receives share_ack, the shared secret becomes fixed.
- **Completeness:** If one honest process completes sharing, all will.

Algorithm 3 $\mathcal{P}_{\text{lb-coin}}^{1:\text{M},\text{acc}} \leq \mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$ on top of $\mathcal{F}_{\text{rbc}}^{\text{acc}}$, and $\mathcal{F}_{\text{hacss}}$: Pseudocode (for process p_i)

```

1: Parameters:
2:   Process  $D$  ▷ the dealer
3: Uses:
4:    $\mathcal{F}_{\text{zkPoK}}^{1:\text{M},\text{acc}}$  ▷ accountable one-to-many zero-knowledge Proof-of-Knowledge
5:    $\mathcal{F}_{\text{rbc}}^{\text{acc}}$  ▷ accountable reliable broadcast
6:    $\mathcal{F}_{\text{hacss}}$  ▷ high dual threshold asynchronous complete secret sharing on  $(\mathbb{Z}_{|\text{Rand}|}, +)$ 
7: Local variables:
8:   Map( $\text{SID} \times \text{Process} \rightarrow \text{Set}(\text{Process})$ )  $\text{RandProviders}_i[]$ 
9:   Map( $\text{SID} \times \text{Process} \rightarrow \text{Set}(\text{Process})$ )  $\text{RandQuorum}_i[]$ 
10:  Map( $\text{SID} \times \text{Process}^2 \rightarrow \text{Rand}$ )  $\text{rand\_material}_i[]$ 
11:  Map( $\text{SID} \times \text{Process} \rightarrow \text{Rand}$ )  $\text{rand\_res}_i[]$ 

12: upon input (toss_req, sid,  $L$ ):
13:   send (share, sid(L,i),  $r_{(L,i)} \xleftarrow{\$} \text{Rand}$ ) to  $\mathcal{F}_{\text{hacss}}$ 
14: upon receiving (share_ack, sid(L,j),  $p_j$ ) from  $\mathcal{F}_{\text{hacss}}$ :
15:    $\text{RandProviders}_i[\text{sid}, L] \leftarrow \text{RandProviders}_i[\text{sid}, L] \cup \{p_j\}$ 
16: upon  $|\text{RandProviders}_i[\text{sid}, L]| = n - t \wedge p_i = L$ :
17:   send (rb_bcast, sid(L,quorum),  $\text{RandProviders}_i[\text{sid}, L]$ ) to  $\mathcal{F}_{\text{rbc}}^{\text{acc}}$ 
18: upon receiving (rb_deliver, sid(L,quorum),  $L, Q_{\text{leader}}$ ) from  $\mathcal{F}_{\text{rbc}}^{\text{acc}}$ :
19:    $\text{RandQuorum}_i[\text{sid}, L] \leftarrow Q_{\text{leader}}$ 
20:   wait until  $\text{RandQuorum}_i[\text{sid}, L] \subseteq \text{RandProviders}_i[\text{sid}, L]$ 
21:   for each  $p_j \in \text{RandQuorum}_i[\text{sid}, L]$ :
22:     send (reconstruct, sid(L,j)) to  $\mathcal{F}_{\text{hacss}}$ 
23: upon receiving (reconstruct_ack, sid(L,j),  $p_j, r_j$ ) from  $\mathcal{F}_{\text{hacss}}$ :
24:    $\text{rand\_material}_i[\text{sid}, L, p_j] \leftarrow r_j$ 
25: upon  $\forall p_j \in \text{RandQuorum}_i[\text{sid}, L], \text{rand\_material}_i[\text{sid}, L, p_j] \neq \perp$ :
26:    $\text{rand\_res}_i[\text{sid}, L] \leftarrow \sum_{p_j \in \text{RandQuorum}_i[\text{sid}, L]} \text{rand\_material}_i[\text{sid}, L, p_j]$ 
27:   if  $p_i = L$ :
28:     send (rb_bcast, sid(L,res),  $\text{rand\_res}_i[\text{sid}, L]$ ) to  $\mathcal{F}_{\text{rbc}}^{\text{acc}}$ 
29: upon receiving (rb_deliver, sid(L,res),  $L, r_L^{\text{sum}}$ ) from  $\mathcal{F}_{\text{rbc}}^{\text{acc}}$  and
30:   wait until  $\text{rand\_res}_i[\text{sid}, L] = r_L^{\text{sum}}$ :
31:     trigger (toss_res, sid,  $L, r_L^{\text{sum}}$ )
32: upon receiving (detection, sid(L,*),  $G, \pi$ ) from  $\mathcal{F}_{\text{rbc}}^{\text{acc}}$ :
33:   trigger (detection, sidL,  $G, \pi$ ) ▷ a disagreement implies  $f > t_e$ 

```

B.2 Pseudocode Description of $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$'s Realization

We now explain the steps of the protocol shown in Algorithm 3.

- (1) **Randomness Sharing.** Upon initiating a coin-toss with leader L , each process generates its own random r_j and shares it using $\mathcal{F}_{\text{hacss}}$. This locks the value while keeping it secret.
- (2) **Acknowledgment Collection.** As share acknowledgments arrive, the process updates $\text{RandProviders}_i[\text{sid}, L]$. Once the leader L has collected $n - t$ acknowledgments, it broadcasts the selected quorum Q_L using $\mathcal{F}_{\text{rbc}}^{\text{acc}}$.
- (3) **Reconstruction.** After receiving Q_L , a process waits until all members of Q_L have shared their secrets (i.e., acknowledged), and then requests their reconstruction. The randomness is computed as the sum of reconstructed values:

$$r = \sum_{p_j \in Q_L} r_j$$

- (4) **Coin Publication.** If the process is the leader, it broadcasts the computed sum using $\mathcal{F}_{\text{rbc}}^{\text{acc}}$. Each participant accepts the result only if it matches their local computation.
- (5) **Accountability.** If any misbehavior occurs during the broadcasts, the accountability guarantees of $\mathcal{F}_{\text{rbc}}^{\text{acc}}$ expose a subset of faulty processes.

THEOREM B.1. $\mathcal{P}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$ (Algorithm 3) UC-realizes $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$ in the $(\mathcal{F}_{\text{rbc}}^{\text{acc}}, \mathcal{F}_{\text{hacss}})$ -hybrid model, in the presence of adaptive adversaries.

INTUITION. *Correctness and randomness:* If at least one r_j is chosen independently and uniformly at random by an honest process, then r is uniformly random due to the additive structure of $(\mathbb{Z}_{|\text{Rand}|}, +)$. The secrecy and binding properties of $\mathcal{F}_{\text{hacss}}$ guarantee that each r_j is fixed before others are revealed.

Liveness: Once $n - t$ correct processes share, completeness of $\mathcal{F}_{\text{hacss}}$ ensures their shares are acknowledged and can be reconstructed. The use of $\mathcal{F}_{\text{rbc}}^{\text{acc}}$ ensures the quorum Q_L and the coin result are reliably and consistently disseminated.

Accountability: Any violation of agreement or premature reconstruction is detected via the accountability mechanisms of $\mathcal{F}_{\text{rbc}}^{\text{acc}}$. In the degraded mode, if two honest processes obtain different outputs, this is caught and the adversary is exposed. $\square$

PROOF. Foreword.

We consider the ideal functionality

$$\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}} = p_1 \mid \dots \mid p_n \mid \tilde{\mathcal{F}}_{\text{lb-coin}}^{1:\text{M},\text{acc}},$$

where $\tilde{\mathcal{F}}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$ corresponds to the ideal functionality $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$ deprived of the dummy processes $p_1, \dots, p_n$.

We work in the $(\mathcal{F}_{\text{rbc}}^{\text{acc}}, \mathcal{F}_{\text{hacss}})$ -hybrid model and analyze the real protocol

$$\mathcal{P}_{\text{lb-coin}}^{1:\text{M},\text{acc}} = \bar{p}_1 \mid \dots \mid \bar{p}_n \mid \mathcal{F}_{\text{rbc}}^{\text{acc}} \mid \mathcal{F}_{\text{hacss}}$$

whose pseudocode is given in Algorithm 3.

Let $\mathcal{A}$ be the adversary of $\bar{\mathcal{P}}$. The simulator $\mathcal{S}$ internally simulates the adversary, denoted by $\mathcal{A}^{\text{sim}}$, and the $\mathcal{F}_{\text{rbc}}^{\text{acc}}$ functionality, denoted by $\mathcal{F}_{\text{rbc}}^{\text{acc},\text{sim}}$. Additionally, upon the corruption of process $\bar{p}_k$, the simulator $\mathcal{S}$ simulates a copy of the corresponding process of the compiled protocol, denoted $\bar{p}_k^{\text{sim}}$, and corrupts the corresponding process p_k .

Messages received by $\mathcal{S}$ from the environment $\mathcal{E}$ are forwarded to $\mathcal{A}^{\text{sim}}$. Similarly, any messages sent by $\mathcal{A}^{\text{sim}}$ that are destined for the environment are forwarded to $\mathcal{E}$. Incoming messages from a corrupted process p_k to $\mathcal{S}$ are forwarded to $\mathcal{A}^{\text{sim}}$ on behalf of the corresponding corrupted process $\bar{p}_k^{\text{sim}}$. Conversely, whenever a corrupted process $\bar{p}_k^{\text{sim}}$ attempts to output a message to the environment, $\mathcal{S}$ instructs the corresponding corrupted process p_k to output the same message to $\mathcal{E}$.

Simulation.

- **Toss Request Handling.** Whenever the simulator $\mathcal{S}$ is notified by $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$ of an input (toss_req, sid, L, p_i), it writes the notification (share_ack, sid_(L,i), p_i) to the leak tape of $\mathcal{A}^{\text{sim}}$ normally connected to $\mathcal{F}_{\text{hacss}}$.
- **Delayed Share Acknowledgment.** Whenever $\mathcal{A}^{\text{sim}}$ instructs $\mathcal{F}_{\text{hacss}}$ to forward the delayed output (share_ack, sid_(L,j), p_j) to p_i , the simulator updates $J_i \leftarrow J_i \cup \{p_j\}$.
- **Malicious Share Submission.** Whenever (share, sid_(L,k), r_k) is sent by $\bar{p}_k^{\text{sim}}$ (under $\mathcal{A}^{\text{sim}}$'s control) to $\mathcal{F}_{\text{hacss}}$, the simulator:
 - records r_k in *rand_material*[sid, L, p_k],
 - and instructs p_k to send (toss_req, sid, L) to $\mathcal{F}_{\text{lb-coin}}^{1:\text{M},\text{acc}}$.

- **Quorum Agreement Handling.** Whenever $(\text{rb_deliver}, \text{sid}_{L, \text{quorum}}, L, Q)$ is sent by $\mathcal{F}_{\text{rbc}}^{\text{acc}, \text{sim}}$ to a correct process, the simulator:
 - stores Q ,
 - and partitions it into disjoint sets H (honest at time of toss request) and B (already corrupted). For each $p_j \in H$, $\mathcal{S}$ updates $\text{rand_material}[\text{sid}, L, p_j]$ with r_j uniformly sampled.
- **Reconstruction Request.** Whenever $Q \cup \{L\} \subseteq J_i$, the simulator writes, for every $p_j \in Q$, the notification $(\text{reconstruct}, \text{sid}_{(L, j)}, p_i)$ to the leak tape of $\mathcal{A}^{\text{sim}}$ connected to $\mathcal{F}_{\text{hacss}}$.
- **Reconstruction Acknowledgment.** Once $(\text{reconstruct}, \text{sid}_{(L, j)}, p_*)$ has been written $n - t$ times, the simulator writes

$$(\text{reconstruct_ack}, \text{sid}_{(L, j)}, p_j, r_j)$$

to $\mathcal{A}^{\text{sim}}$'s tape, where $r_j = \text{rand_material}[\text{sid}, L, p_j]$

- **Share Reconstruction by Receiver.** Whenever $\mathcal{A}^{\text{sim}}$ instructs $\mathcal{F}_{\text{hacss}}$ to send the delayed output

$$(\text{reconstruct_ack}, \text{sid}_{(L, j)}, p_j, r_j)$$

to p_i for $p_j \in Q$, the simulator updates $\text{cnt}_i \leftarrow \text{cnt}_i + 1$, initially set at 0.

- **Final Coin Output.** When $\mathcal{F}_{\text{rbc}}^{\text{acc}, \text{sim}}$ sends $(\text{rb_deliver}, \text{sid}_{L, \text{res}}, L, r^{\text{sum}})$ to p_i , the simulator waits until:

$$\text{cnt}_i = |Q| \quad \text{and} \quad r^{\text{sum}} = \sum_{p_j \in Q} \text{rand_material}[\text{sid}, L, p_j],$$

and then instructs $\mathcal{F}_{\text{lb-coin}}^{1:\text{M}, \text{acc}}$ to send $(\text{toss_res}, \text{sid}, L, r^{\text{sum}})$ to p_i .

- **Accountability.** Any accountability evidence produced by $\mathcal{F}_{\text{rbc}}^{\text{acc}, \text{sim}}$ is forwarded verbatim.

Indistinguishability argument. We have defined a perfect forward simulation from $\mathcal{A} \mid \mathcal{P}_{\text{lb-coin}}^{1:\text{M}, \text{acc}}$ to $\mathcal{S} \mid \mathcal{F}_{\text{lb-coin}}^{1:\text{M}, \text{acc}}$. The binding property of $\mathcal{F}_{\text{hacss}}$ ensures that the values

$$r_H = \sum_{p_i \in H} \text{rand_material}[\text{sid}, L, p_i] \quad \text{and} \quad r_B = \sum_{p_k \in B} \text{rand_material}[\text{sid}, L, p_k]$$

are independent. Moreover, whenever $f < n - t$, the honest set H is non-empty, so r_H is uniformly distributed. As a result, the output value $r^* = r_H + r_B$ returned by $\mathcal{F}_{\text{lb-coin}}^{1:\text{M}, \text{acc}}$ is also uniformly distributed.

Liveness. Completeness of $\mathcal{F}_{\text{hacss}}$ and totality of $\mathcal{F}_{\text{rbc}}^{\text{acc}}$ guarantee the termination of the protocol under a correct leader, and totality in general.

Accountability. If agreement is violated, it implies that the agreement property of the underlying $\mathcal{F}_{\text{rbc}}^{\text{acc}}$ has been breached. In such a case, a detection is triggered (see Algorithm 3) and propagated to the higher level.

□

Comment on the Case $f \geq n - t$.

Recall that the random tape r referenced in the commit-and-prove functionality is composed as $r = r^{\text{com}} + r^{\text{coin}}$, where r^{com} is the committed secret randomness provided during the commitment phase, and r^{coin} is the output of the leader-based coin-tossing functionality, with the leader being the owner of the random tape.

When $f \geq n - t$, a malicious leader may bias both components r^{com} and r^{coin} , thereby gaining full control over its random tape r . To prevent such bias, one may resort to a setup phase assuming

a synchronous network with a conservative upper bound on message delivery time. In this case, two alternatives are available:

- Use an accountable synchronous MPC protocol [112] to jointly generate random tapes with strong bias-resistance and accountability.
- Derive entropy from an external public source, such as the hash of a blockchain block, assuming its unpredictability and responsiveness.

C ZKPOK: REALIZING $\mathcal{F}_{\text{zkPoK}}^{1:M}$

C.1 Universally Composable Zero-knowledge Proof-of-Knowledge

Goldwasser, Micali, and Rackoff introduced zero-knowledge proofs that enable a prover to convince a verifier that a NP-statement is true without revealing anything else [85]. They have three core properties:

- Completeness: Given a statement and a witness, the prover can convince the verifier.
- Soundness: A malicious prover cannot convince the verifier of a false statement.
- Zero-knowledge: The proof does not reveal anything but the truth of the statement, in particular it does not reveal the prover’s witness.

A zero-knowledge proof-of-knowledge has the additional property that the prover must “know” the corresponding witness [20]. This corresponds to:

- Knowledge Extractability: Take any (possibly malicious) prover algorithm, that convinces the verifier with non-negligible probability. Then there is an *extractor* that interacts with this prover and recovers a witness that the statement is true.

A protocol that adaptively (and interactively) UC-realizes the corresponding ideal functionality $\mathcal{F}_{\text{zkPoK}}$ (see Functionality 5) was shown in [43], in the common reference string (CRS) model. The functionality $\mathcal{F}_{\text{zkPoK}}$ implicitly states the usual notions of completeness, soundness, knowledge extractability, and zero-knowledge.

Functionality 5 $\mathcal{F}_{\text{zkPoK}}$

Parameters:

$\mathcal{R}$: An NP relation for an NP language.

Events:

request (prove, sid, V , x , w): Proving to V that one knows a witness w for the statement x .

notification (verification, sid, P , x): Verifying that the prover P knows a witness for x .

Functionality:

Upon receiving (prove, sid, V , x , w) from P with $\mathcal{R}(x, w) = 1$:

send (verification, sid, P , x) to Sim and V as a delayed output.

A stronger version of ZKPoK is its non-interactive counterpart NIZKPoK. Here, the prover can generate a *proof* which is “self-certifying”. The corresponding functionality $\mathcal{F}_{\text{nizkPoK}}$ is given in Functionality 6. A recent adaptively-secure UC-realization of $\mathcal{F}_{\text{nizkPoK}}$ has been given [47] in the common reference string (CRS) model. When succinctness is achieved, meaning, for any $(x, w) \in \mathcal{R}$, the length of the proof is in $\text{poly}(\kappa, \log(|w|), \log(|x|))$, the NIZKPoK is called a zkSNARK. It has recently been shown [51] that the Micali construction [104] is adaptively UC-secure in the programmable random oracle model (GROM in the UC). Another UC-realization of zkSNARK has been recently proposed [79] with a global random oracle that is observable but not programmable.

Functionality 6 $\mathcal{F}_{\text{nizkPoK}}$ **Parameters:**

$\mathcal{R}$: An NP relation for an NP language.

Local state:

Q: A proof table, initially empty.

Events:

request (prove, sid, x , w): Requesting a NIZKPoK for the statement x from witness w .

notification (proof, sid, π): Getting a NIZKPoK π for the statement x .

request (verify, sid, x , π): Verifying the knowledge of a witness for statement x .

Functionality:

Upon receiving (prove, sid, x , w) from process p_i :

If $(x, w) \notin \mathcal{R}$, ignore the message.

Otherwise, send (prove, sid, x) to Sim.

Upon receiving (proof, sid, π) from Sim:

Store (x, π) in $Q[\text{sid}]$.

Send (proof, sid, π) to p_i .

Upon receiving (verify, sid, x , π) from process p_i :

If $(x, \pi) \notin Q[\text{sid}]$, send (verify, sid, x , π) to Sim.

Upon receiving (witness, w) from Sim:

If $(x, w) \in \mathcal{R}$, store (x, π) in $Q[\text{sid}]$.

Return (verification, sid, $(x, \pi) \in ?Q[\text{sid}]$) to p_i .

C.2 Achieving One-to-Many ZKPoK

The functionalities $\mathcal{F}_{\text{zkPoK}}$ and $\mathcal{F}_{\text{nizkPoK}}$ can be used as building blocks of the functionality $\mathcal{F}_{\text{zkPoK}}^{1:M}$ (see Functionality 8), the *one-to-many* version of zero-knowledge proof-of-knowledge. The completeness becomes, given a statement and a witness, the prover can convince all the verifiers in Ψ . In addition, the functionality inherits from the totality and agreement property of the reliable-broadcast functionality $\mathcal{F}_{\text{rbc}}$ (see Functionality 7). This means (1) no two correct processes trigger two different verification notifications for the same session ID, and (2) if a correct process triggers a verification notification for a certain session ID, then all correct processes eventually do the same.

Functionality 7 $\mathcal{F}_{\text{rbc}}$ **Events:**

request (rb_bcast, sid, m): Reliable-broadcasting the message m .

notification (rb_deliver, sid, P , m): Reliable-delivering the message m .

Functionality:

Upon receiving (rb_bcast, sid, m) from the source P for the first time for session ID sid:

Send (rb_deliver, sid, P , m) to Sim and as a delayed output to each $p_j \in \Psi$.

Now, we discuss the realization of $\mathcal{F}_{\text{zkPoK}}^{1:M}$, on top of $\mathcal{F}_{\text{rbc}}$.

C.2.1 $\mathcal{F}_{\text{zkPoK}}^{1:M}$ on top of $\mathcal{F}_{\text{nizkPoK}}$ and $\mathcal{F}_{\text{rbc}}$. The first solution (Algorithm 4) uses the non-interactive ZKPoK functionality $\mathcal{F}_{\text{nizkPoK}}$ parametrized with an NP relation $\mathcal{R}$ (line 2), and the reliable broadcast functionality $\mathcal{F}_{\text{rbc}}$ (line 3). Upon a prover's request to prove a statement x with a witness w , the request is sent to $\mathcal{F}_{\text{nizkPoK}}$ (line 6, line 7). Upon receiving a proof (x, π) from $\mathcal{F}_{\text{nizkPoK}}$, the proof is broadcast using $\mathcal{F}_{\text{rbc}}$ (line 8, and 9). When a process delivers (x, π) from $\mathcal{F}_{\text{rbc}}$, it verifies the proof by sending it back to $\mathcal{F}_{\text{nizkPoK}}$ (line 10, and 11). Upon successful verification, the process triggers the corresponding verification notification to confirm the statement x (line 12, and 13).

Functionality 8 $\mathcal{F}_{\text{zkPoK}}^{1:M}$ **Parameters:** $\mathcal{R}$: An NP relation for an NP language.**Events:***request* (prove, sid, x , w): Proving one knows a witness w for the statement x .*notification* (verification, sid, P , x): Verifying the prover P knows a witness for x .**Functionality:**Upon receiving (prove, sid, x , w) from P for the first time for sid, s.t. $(x, w) \in \mathcal{R}$:Send (verification, sid, P , x) to Sim and as a delayed output to each $p_j \in \Psi$.**Algorithm 4** $\mathcal{P}_{\text{zkPoK}}^{1:M} \leq \mathcal{F}_{\text{zkPoK}}^{1:M}$ on top of $\mathcal{F}_{\text{nizkPoK}}$ and $\mathcal{F}_{\text{rbc}}$: Pseudocode (for process p_i)

- 1: **Uses:**
- 2: $\mathcal{F}_{\text{nizkPoK}}$ parametrized with an NP-relation $\mathcal{R}$ ▶ non-interactive zero-knowledge Proof-of-Knowledge
- 3: $\mathcal{F}_{\text{rbc}}$ ▶ reliable broadcast
- 4: **Parameters:**
- 5: P : prover
- 6: **upon** requesting (prove, sid, ssid, x , w):
- 7: **send** (prove, sid, ssid, x , w) to $\mathcal{F}_{\text{nizkPoK}}$
- 8: **upon** receiving (proof, sid, ssid, x , π) from $\mathcal{F}_{\text{nizkPoK}}$:
- 9: **send** (rb_bcast, sid, ssid, (x, π)) to $\mathcal{F}_{\text{rbc}}$
- 10: **upon** receiving (rb_deliver, sid, ssid, P , x , π) from $\mathcal{F}_{\text{rbc}}$:
- 11: **send** (verify, sid, ssid, (x, π)) to $\mathcal{F}_{\text{nizkPoK}}$
- 12: **upon** receiving (verification, sid, ssid, ' $(x, \pi) \in Q$ ') from $\mathcal{F}_{\text{nizkPoK}}$:
- 13: **trigger** (verification, sid, ssid, P , x)

C.2.2 $\mathcal{F}_{\text{zkPoK}}^{1:M}$ on top of $\mathcal{F}_{\text{zkPoK}}$, $\mathcal{F}_{\text{rbc}}$, and certificates. To highlight the modularity of our approach, we point out another realization, inspired from [91], that does not leverage the non-interactivity of NIZKPoK.

Algorithm 5 uses a (potentially interactive) ZKPoK functionality $\mathcal{F}_{\text{zkPoK}}$ (line 2), the reliable broadcast functionality $\mathcal{F}_{\text{rbc}}$ (line 3), and a threshold signature functionality $\mathcal{F}_{\text{tsig}}$ (line 4). The first adaptively-secure threshold signature scheme has been recently presented in [70]. We conjecture that, by an argument similar to the proof of [36, Theorem 2], a threshold signature scheme satisfying unforgeability and robustness under chosen-message attacks, such as the one of [70], can be used to UC-realize the corresponding ideal functionality $\mathcal{F}_{\text{tsig}}$.

When a prover initiates a request to prove a statement x with a witness w , it sends individual requests to each verifier using $\mathcal{F}_{\text{zkPoK}}$ (line 8–10). Each verifier receiving a notification from $\mathcal{F}_{\text{zkPoK}}$, supporting statement x , generates a partial signature for x using $\mathcal{F}_{\text{tsig}}$ and sends the partial signature back to the prover via $\mathcal{F}_{\text{auth}}$ (line 11–14). The prover checks the validity of each partial-signature, using $\mathcal{F}_{\text{tsig}}$. (line 15–16). If the prover confirms the validity of the partial signature, it adds the signature to a set S (line 17–18). When the prover collects $n - t$ valid partial signatures, it combines them into a threshold signature using $\mathcal{F}_{\text{tsig}}$ (line 19–20). The prover broadcasts the threshold signature via $\mathcal{F}_{\text{rbc}}$ (line 21–22). Upon receiving the threshold signature from $\mathcal{F}_{\text{rbc}}$, verifiers verify the threshold signature using $\mathcal{F}_{\text{tsig}}$ (line 23–24). If the verification succeeds, the statement is confirmed, and the process triggers the verification output (line 25).

Although this second realization operates with weaker cryptographic assumptions, it comes with a trade-off: soundness (and knowledge extractability) is compromised when $f \geq n - t$, as a

Algorithm 5 $\mathcal{P}_{\text{zkPoK}}^{1:M} \leq \mathcal{F}_{\text{zkPoK}}^{1:M}$ on top of $\mathcal{F}_{\text{zkPoK}}$, $\mathcal{F}_{\text{rbc}}$, and $\mathcal{F}_{\text{tss}}$: Pseudocode (for process p_i)

```

1: Uses:
2:    $\mathcal{F}_{\text{zkPoK}}$  parametrized with an NP relation  $\mathcal{R}$       ▶ (potentially interactive) zero-knowledge Proof-of-Knowledge
3:    $\mathcal{F}_{\text{rbc}}$                                              ▶ reliable broadcast
4:    $\mathcal{F}_{\text{tss}}$                                              ▶ threshold signature scheme
5:    $\mathcal{F}_{\text{auth}}$                                            ▶ authenticated network
6: Parameters:
7:    $P$ : prover
8: upon requesting (prove, sid, ssid,  $x$ ,  $w$ ):
9:   for each  $p_j \in \Psi$ :
10:    send (prove, sid, ssid,  $p_j$ ,  $x$ ,  $w$ ) to  $\mathcal{F}_{\text{zkPoK}}$ 
11: upon receiving (proof, sid, ssid,  $P$ ,  $x$ ) from  $\mathcal{F}_{\text{zkPoK}}$ :
12:   send (share_sign, sid, ssid,  $\langle \text{ACK}, x \rangle$ ) to  $\mathcal{F}_{\text{tss}}$ 
13: upon receiving (share_signature, sid, ssid,  $\sigma_j$ ,  $\langle \text{ACK}, x \rangle$ ) from  $\mathcal{F}_{\text{tss}}$ :
14:   send (send, sid, ssid,  $P$ ,  $\sigma_j$ ,  $\langle \text{ACK}, x \rangle$ ) to  $\mathcal{F}_{\text{auth}}$ 
15: upon receiving (rcv, sid, ssid,  $p_j$ ,  $\sigma_j$ ,  $\langle \text{ACK}, x \rangle$ ) from  $\mathcal{F}_{\text{auth}}$ :
16:   send (share_verify, sid, ssid,  $p_j$ ,  $\sigma_j$ ,  $\langle \text{ACK}, x \rangle$ ) to  $\mathcal{F}_{\text{tss}}$ 
17: upon receiving (share_verification, sid, ssid,  $p_j$ ,  $\sigma_j$ ,  $\langle \text{ACK}, x \rangle$ , true) from  $\mathcal{F}_{\text{tss}}$ :
18:    $S \leftarrow S \cup \{(\sigma_j, \langle \text{ACK}, x \rangle)\}$ 
19: upon  $|S| = n - t$ :
20:   send (combine, sid, ssid,  $S$ ) to  $\mathcal{F}_{\text{tss}}$ 
21: upon receiving (th_signature, sid, ssid,  $\sigma^T$ ,  $\langle \text{ACK}, x \rangle$ ) from  $\mathcal{F}_{\text{tss}}$ :
22:   send (rb_bcast, sid, ssid,  $(\sigma^T, \langle \text{ACK}, x \rangle)$ ) to  $\mathcal{F}_{\text{rbc}}$ 
23: upon receiving (rb_deliver, sid, ssid,  $P$ ,  $(\sigma^T, \langle \text{ACK}, x \rangle)$ ) from  $\mathcal{F}_{\text{rbc}}$ :
24:   send (th_verify, sid, ssid,  $\sigma^T$ ,  $\langle \text{ACK}, x \rangle$ ) to  $\mathcal{F}_{\text{tss}}$ 
25: upon receiving (th_verification, sid, ssid,  $\sigma^T$ ,  $\langle \text{ACK}, x \rangle$ , true) from  $\mathcal{F}_{\text{tss}}$ :
26:   trigger (verification, sid, ssid,  $P$ ,  $x$ )

```

coalition of $n - t$ malicious processes could forge the certificate. Consequently, protocols based on this realization require $t_{\text{acc}} < n - t$, typically setting $t_{\text{acc}} = \lceil 2n/3 \rceil - 1$.

D THE PUBLIC JUDGE AND THE JUDICIAL FUNCTIONALITY

We specify the public judge Judge (Global Functionality 11) and the auxiliary judicial functionality $\mathcal{F}_{\text{judi}}$ (Functionality 9–10); their parallel composition yields the globally available functionality $\mathcal{J} \triangleq \text{Judge} \mid \mathcal{F}_{\text{judi}}$. The functionality $\mathcal{F}_{\text{judi}}$ is the multi-signature analog of the certification functionality $\mathcal{F}_{\text{cert}}$ and is realized from a (multi-)signature functionality $\mathcal{F}_{(\text{m})\text{sig}}$ together with a certification authority (PKI) $\mathcal{F}_{\text{CA}}$, following [36].

On a request (verdict, sid, (m^1, Q^1, σ^1) , (m^2, Q^2, σ^2)), Judge queries $\mathcal{F}_{\text{judi}}$ to verify that, for each $i \in \{1, 2\}$, σ^i is a valid multi-signature by Q^i on m^i ; if both verify and the messages conflict (e.g., same header, distinct payload), it returns the verdict $\text{dis}(Q^1 \cap Q^2)$. Making Judge globally accessible captures *external verifiability*: third parties that did not participate in the execution can validate evidence.

The same interface is used to validate BreakAgreement requests: before an accountable functionality allows an agreement break, it checks that the attached proof (obtainable by the ideal adversary via $\mathcal{F}_{\text{judi}}$) incriminates at least d processes. In our construction, accountability reduces to the accountable agreement of the confirmer $\mathcal{F}_{\text{conf}}^{\text{acc}}$; intermediate layers merely forward proofs of misbehavior (and derived verdicts) upward. We omit evidence-collection details at those layers, as they arise from the concrete realization of $\mathcal{F}_{\text{conf}}^{\text{acc}}$ in [53].

Functionality 9 $\mathcal{F}_{\text{judi}}$ – Signing, Aggregating

//A certification authority functionality $\mathcal{F}_{\text{ca}}$ allows processes to agree on one unique public key per process

Signature Generation:

Upon receiving (sign, sid, p_i , m) from p_i :
 Forward (sign, sid, p_i , m) to the adversary.
 Upon receiving (signature, sid, p_i , m , σ_i) from the adversary:
 If no entry (sid, m , σ_i , 0) is recorded:
 Record the entry (sid, m , σ_i , $\{p_i\}$, 1).
 Output (signature, sid, p_i , m , σ_i) to p_i .

Signature Combination:

Upon receiving (cb, sid, G , m , $\{\sigma_j\}_{p_j \in G}$), from p_i , s.t. $G \subseteq \Psi \wedge |G| \geq 2$:
 Forward (cb, sid, G , m , $\{\sigma_j\}_{p_j \in G}$) to the adversary.
 Upon receiving (cb, sid, G , m , $\{(\phi_j)\}_{p_j \in G}$, σ_G) from the adversary:
 Determine $f_G \leftarrow \text{BatchIndividualVerif}(\text{sid}, G, m, \{(\sigma_j, \phi_j)\}_{p_j \in G})$
 If $f_G = 1$:
 Record the entry (sid, m , σ_G , G , 1).
 Output (multisignature, sid, G , m , σ_G) to p_i .
 Else:
 Output (multisignature, sid, G , m , $\perp$) to p_i .

Functionality 10 $\mathcal{F}_{\text{judi}}$ – Verifying

Signature Verification:

Upon receiving (verify, sid, G , m , σ_G), s.t. $G \subseteq \Psi \wedge |G| \geq 1$:
 Forward (verify, sid, G , m , σ_G) to the adversary.
 Upon receiving (verified, sid, G , m , ϕ_G) from the adversary:
 Determine $f_G \leftarrow \text{Verif}(\text{sid}, G, m, \sigma_G, \phi_G)$
 Output (verified, sid, G , m , σ_G , f_G) to the requester (process or judge).

Verification Procedure:

Verif(sid, G , m , σ , ϕ_G):
 (1) If the entry (sid, m , σ , G , 1) is recorded: return $f = 1$.
 (Completeness: Verification succeeds for valid (multi-)signatures.)
 (2) Else, if $\exists p_j \in G \cap \text{Correct}$, and no entry (sid, m , σ' , $G' \ni p_i$, 1) is recorded:
 record the entry (sid, m , σ , G , 0) and return $f = 0$.
 (Unforgeability: Verification fails if one signer did not sign m .)
 (3) Else, if an entry (sid, m , σ , G , f') is recorded: return $f = f'$.
 (Consistency: Identical parameters produce the same verification result.)
 (4) Else: record the entry (sid, m , σ , G , ϕ_G) and return $f = \phi_G$.

BatchIndividualVerif(sid, G , m , $\{(\sigma_j, \phi_j)\}_{p_j \in G}$):
 Determine $\{f_j \leftarrow \text{Verif}(\text{sid}, \{p_j\}, m, \sigma_j, \phi_j)\}_{p_j \in G}$
 Return $\bigwedge_{p_j \in G} f_j$

Functionality 11 Judge – Globally available

Verdict request:

Upon receiving (verdict, sid, (m^1, Q^1, σ^1) , (m^2, Q^2, σ^2)) from a requester R :
 $\forall i \in \{1, 2\}$: send (verify, sid, Q^i , m^i , σ^i) to $\mathcal{F}_{\text{judi}}$
 When (verified, sid, Q^i , m^i , σ^i , f^i) is received from $\mathcal{F}_{\text{judi}}$ for each $i \in \{1, 2\}$:
 if $m^1.\text{header} = m^2.\text{header} \wedge m^1.\text{payload} \neq m^2.\text{payload} \wedge f^1 = f^2 = \text{true}$:
 send (verdict, sid, $\text{dis}(Q^1 \cap Q^2)$) to R . (**equivocation detected**)
 else
 send (verdict, sid, $\perp$) to R . (**insufficient evidence**)

REFERENCES

- [1] ABRAHAM, I., CHAN, T. H., DOLEV, D., NAYAK, K., PASS, R., REN, L., AND SHI, E. Communication Complexity of Byzantine Agreement, Revisited. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019* (2019), P. Robinson and F. Ellen, Eds., ACM, pp. 317–326.
- [2] ABRAHAM, I., CHAN, T. H., DOLEV, D., NAYAK, K., PASS, R., REN, L., AND SHI, E. Communication complexity of byzantine agreement, revisited. *Distributed Comput.* 36, 1 (2023), 3–28.
- [3] ALHADDAD, N., DAS, S., DUAN, S., REN, L., VARIA, M., XIANG, Z., AND ZHANG, H. Balanced byzantine reliable broadcast with near-optimal communication and improved computation. In *PODC '22: ACM Symposium on Principles of Distributed Computing, Salerno, Italy, July 25 - 29, 2022* (2022), A. Milani and P. Woelfel, Eds., ACM, pp. 399–417.
- [4] ALHADDAD, N., VARIA, M., AND YANG, Z. Haven++: Batched and packed dual-threshold asynchronous complete secret sharing with applications. *IACR Commun. Cryptol.* 1, 4 (2024), 3.
- [5] ANTA, A. F., KONWAR, K. M., GEORGIU, C., AND NICOLAOU, N. C. Formalizing and implementing distributed ledger objects. *SIGACT News* 49, 2 (2018), 58–76.
- [6] ATTIYA, H., AND WELCH, J. L. *Distributed computing - fundamentals, simulations, and advanced topics* (2. ed.). Wiley series on parallel and distributed computing. Wiley, 2004.
- [7] AUMANN, Y., AND LINDELL, Y. Security against covert adversaries: Efficient protocols for realistic adversaries. In *Theory of Cryptography, 4th Theory of Cryptography Conference, TCC 2007, Amsterdam, The Netherlands, February 21-24, 2007, Proceedings* (2007), S. P. Vadhan, Ed., vol. 4392 of *Lecture Notes in Computer Science*, Springer, pp. 137–156.
- [8] AUMANN, Y., AND LINDELL, Y. Security against covert adversaries: Efficient protocols for realistic adversaries. *J. Cryptol.* 23, 2 (2010), 281–343.
- [9] BACKES, M., BENDUN, F., CHOUDHURY, A., AND KATE, A. Asynchronous MPC with a strict honest majority using non-equivocation. In *ACM Symposium on Principles of Distributed Computing, PODC '14, Paris, France, July 15-18, 2014* (2014), M. M. Halldórsson and S. Dolev, Eds., ACM, pp. 10–19.
- [10] BACKES, M., DRUSCHEL, P., HAEBERLEN, A., AND UNRUH, D. CSAR: A practical and provable technique to make randomized systems accountable. In *Proceedings of the Network and Distributed System Security Symposium, NDSS 2009, San Diego, California, USA, 8th February - 11th February 2009* (2009), The Internet Society.
- [11] BACKES, M., HOFHEINZ, D., MÜLLER-QUADE, J., AND UNRUH, D. On fairness in simulatability-based cryptographic systems. In *Proceedings of the 2005 ACM workshop on Formal methods in security engineering, FMSE 2005, Fairfax, VA, USA, November 11, 2005* (2005), V. Atluri, P. Samarati, R. Küsters, and J. C. Mitchell, Eds., ACM, pp. 13–22.
- [12] BACKES, M., PFITZMANN, B., STEINER, M., AND W Aidner, M. Polynomial fairness and liveness. In *15th IEEE Computer Security Foundations Workshop (CSFW-15 2002), 24-26 June 2002, Cape Breton, Nova Scotia, Canada* (2002), IEEE Computer Society, pp. 160–174.
- [13] BACKES, M., PFITZMANN, B., AND W Aidner, M. A composable cryptographic library with nested operations. In *Proceedings of the 10th ACM Conference on Computer and Communications Security, CCS 2003, Washington, DC, USA, October 27-30, 2003* (2003), S. Jajodia, V. Atluri, and T. Jaeger, Eds., ACM, pp. 220–230.
- [14] BACKES, M., PFITZMANN, B., AND W Aidner, M. A general composition theorem for secure reactive systems. In *Theory of Cryptography, First Theory of Cryptography Conference, TCC 2004, Cambridge, MA, USA, February 19-21, 2004, Proceedings* (2004), M. Naor, Ed., vol. 2951 of *Lecture Notes in Computer Science*, Springer, pp. 336–354.
- [15] BACKES, M., PFITZMANN, B., AND W Aidner, M. Polynomial liveness. *J. Comput. Secur.* 12, 3-4 (2004), 589–617.
- [16] BARTHE, G., CEDERQUIST, J., AND TARENTO, S. A machine-checked formalization of the generic model and the random oracle model. In *Automated Reasoning - Second International Joint Conference, IJCAR 2004, Cork, Ireland, July 4-8, 2004, Proceedings* (2004), D. A. Basin and M. Rusinowitch, Eds., vol. 3097 of *Lecture Notes in Computer Science*, Springer, pp. 385–399.
- [17] BAUM, C., MELISSARIS, N., RACHURI, R., AND SCHOLL, P. Cheater identification on a budget: MPC with identifiable abort from pairwise macs. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VIII* (2024), L. Reyzin and D. Stebila, Eds., vol. 14927 of *Lecture Notes in Computer Science*, Springer, pp. 454–488.
- [18] BAUM, C., ORSINI, E., AND SCHOLL, P. Efficient secure multiparty computation with identifiable abort. In *Theory of Cryptography - 14th International Conference, TCC 2016-B, Beijing, China, October 31 - November 3, 2016, Proceedings, Part I* (2016), M. Hirt and A. D. Smith, Eds., vol. 9985 of *Lecture Notes in Computer Science*, pp. 461–490.
- [19] BAUM, C., ORSINI, E., SCHOLL, P., AND SORIA-VAZQUEZ, E. Efficient constant-round MPC with identifiable abort and public verifiability. In *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part II* (2020), D. Micciancio and T. Ristenpart, Eds., vol. 12171 of *Lecture Notes in Computer Science*, Springer, pp. 562–592.
- [20] BELLARE, M., AND GOLDBREICH, O. On defining proofs of knowledge. In *Advances in Cryptology - CRYPTO '92, 12th Annual International Cryptology Conference, Santa Barbara, California, USA, August 16-20, 1992, Proceedings* (1992), E. F. Brickell, Ed., vol. 740 of *Lecture Notes in Computer Science*, Springer, pp. 390–420.

- [21] BEN-DAVID, N., CHAN, B. Y., AND SHI, E. Revisiting the power of non-equivocation in distributed protocols. In *PODC '22: ACM Symposium on Principles of Distributed Computing, Salerno, Italy, July 25 - 29, 2022* (2022), A. Milani and P. Woelfel, Eds., ACM, pp. 450–459.
- [22] BEN-OR, M., KELMER, B., AND RABIN, T. Asynchronous Secure Computations with Optimal Resilience. In *Proceedings of the thirteenth annual ACM symposium on Principles of distributed computing* (1994), pp. 183–192.
- [23] BHADAURIA, R., AND CHOUDHURY, A. Brief announcement: Asynchronous secure distributed computing with transferrable non-equivocation revisited. In *Proceedings of the 2018 ACM Symposium on Principles of Distributed Computing, PODC 2018, Egham, United Kingdom, July 23-27, 2018* (2018), C. Newport and I. Keidar, Eds., ACM, pp. 265–267.
- [24] BLANCHET, B. A computationally sound mechanized prover for security protocols. *IEEE Trans. Dependable Secur. Comput.* 5, 4 (2008), 193–207.
- [25] BLUM, E., KATZ, J., LIU-ZHANG, C., AND LOSS, J. Asynchronous byzantine agreement with subquadratic communication. In *Theory of Cryptography - 18th International Conference, TCC 2020, Durham, NC, USA, November 16-19, 2020, Proceedings, Part I* (2020), R. Pass and K. Pietrzak, Eds., vol. 12550 of *Lecture Notes in Computer Science*, Springer, pp. 353–380.
- [26] BRACHA, G. An Asynchronous $[(n-1)/3]$ -Resilient Consensus Protocol. In *Proceedings of the Third Annual ACM Symposium on Principles of Distributed Computing, Vancouver, B. C., Canada, August 27-29, 1984* (1984), T. Kameda, J. Misra, J. G. Peters, and N. Santoro, Eds., ACM, pp. 154–162.
- [27] BRACHA, G. Asynchronous byzantine agreement protocols. *Inf. Comput.* 75, 2 (1987), 130–143.
- [28] BRACHA, G., AND TOUEG, S. Resilient consensus protocols. In *Proceedings of the Second Annual ACM Symposium on Principles of Distributed Computing, Montreal, Quebec, Canada, August 17-19, 1983* (1983), R. L. Probert, N. A. Lynch, and N. Santoro, Eds., ACM, pp. 12–26.
- [29] CACHIN, C., KURSAWE, K., PETZOLD, F., AND SHOUP, V. Secure and efficient asynchronous broadcast protocols. In *Advances in Cryptology - CRYPTO 2001, 21st Annual International Cryptology Conference, Santa Barbara, California, USA, August 19-23, 2001, Proceedings* (2001), J. Kilian, Ed., vol. 2139 of *Lecture Notes in Computer Science*, Springer, pp. 524–541.
- [30] CACHIN, C., KURSAWE, K., AND SHOUP, V. Random oracles in constantinople: practical asynchronous byzantine agreement using cryptography (extended abstract). In *Proceedings of the Nineteenth Annual ACM Symposium on Principles of Distributed Computing, July 16-19, 2000, Portland, Oregon, USA* (2000), G. Neiger, Ed., ACM, pp. 123–132.
- [31] CACHIN, C., KURSAWE, K., AND SHOUP, V. Random Oracles in Constantinople: Practical Asynchronous Byzantine Agreement Using Cryptography. *J. Cryptol.* 18, 3 (2005), 219–246.
- [32] CAMAIONI, M., GUERRAUI, R., KOMATOVIC, J., MONTI, M., ROMAN, P., VIDIGUEIRA, M., AND VORON, G. Carbon: Scaling trusted payments with untrusted machines. *IEEE Trans. Dependable Secur. Comput.* 22, 2 (2025), 1168–1180.
- [33] CAMENISCH, J., KRENN, S., KÜSTERS, R., AND RAUSCH, D. iuc: Flexible universal composability made simple. In *Advances in Cryptology - ASIACRYPT 2019 - 25th International Conference on the Theory and Application of Cryptology and Information Security, Kobe, Japan, December 8-12, 2019, Proceedings, Part III* (2019), S. D. Galbraith and S. Moriai, Eds., vol. 11923 of *Lecture Notes in Computer Science*, Springer, pp. 191–221.
- [34] CANETTI, R. Universally composable security: A new paradigm for cryptographic protocols. In *42nd Annual Symposium on Foundations of Computer Science, FOCS 2001, 14-17 October 2001, Las Vegas, Nevada, USA* (2001), IEEE Computer Society, pp. 136–145.
- [35] CANETTI, R. Universally composable signatures, certification and authentication. *IACR Cryptol. ePrint Arch.* (2003), 239.
- [36] CANETTI, R. Universally composable signature, certification, and authentication. In *17th IEEE Computer Security Foundations Workshop, (CSFW-17 2004), 28-30 June 2004, Pacific Grove, CA, USA* (2004), IEEE Computer Society, p. 219.
- [37] CANETTI, R. Universally composable security. *J. ACM* 67, 5 (2020), 28:1–28:94.
- [38] CANETTI, R., CHEUNG, L., KAYNAR, D. K., LISKOV, M. D., LYNCH, N. A., PEREIRA, O., AND SEGALA, R. Time-bounded task-pioas: A framework for analyzing security protocols. In *Distributed Computing, 20th International Symposium, DISC 2006, Stockholm, Sweden, September 18-20, 2006, Proceedings* (2006), S. Dolev, Ed., vol. 4167 of *Lecture Notes in Computer Science*, Springer, pp. 238–253.
- [39] CANETTI, R., CHEUNG, L., KAYNAR, D. K., LISKOV, M. D., LYNCH, N. A., PEREIRA, O., AND SEGALA, R. Analyzing security protocols using time-bounded task-pioas. *Discret. Event Dyn. Syst.* 18, 1 (2008), 111–159.
- [40] CANETTI, R., CHEUNG, L., KAYNAR, D. K., LYNCH, N. A., AND PEREIRA, O. Compositional security for task-pioas. In *20th IEEE Computer Security Foundations Symposium, CSF 2007, 6-8 July 2007, Venice, Italy* (2007), IEEE Computer Society, pp. 125–139.
- [41] CANETTI, R., CHEUNG, L., KAYNAR, D. K., LYNCH, N. A., AND PEREIRA, O. Modeling computational security in long-lived systems. In *CONCUR 2008 - Concurrency Theory, 19th International Conference, CONCUR 2008, Toronto, Canada, August 19-22, 2008. Proceedings* (2008), F. van Breugel and M. Chechik, Eds., vol. 5201 of *Lecture Notes in Computer*

Science, Springer, pp. 114–130.

- [42] CANETTI, R., DODIS, Y., PASS, R., AND WALFISH, S. Universally composable security with global setup. In *Theory of Cryptography, 4th Theory of Cryptography Conference, TCC 2007, Amsterdam, The Netherlands, February 21-24, 2007, Proceedings* (2007), S. P. Vadhan, Ed., vol. 4392 of *Lecture Notes in Computer Science*, Springer, pp. 61–85.
- [43] CANETTI, R., AND FISCHLIN, M. Universally composable commitments. In *Advances in Cryptology - CRYPTO 2001, 21st Annual International Cryptology Conference, Santa Barbara, California, USA, August 19-23, 2001, Proceedings* (2001), J. Kilian, Ed., vol. 2139 of *Lecture Notes in Computer Science*, Springer, pp. 19–40.
- [44] CANETTI, R., LINDELL, Y., OSTROVSKY, R., AND SAHAI, A. Universally composable two-party and multi-party secure computation. In *Proceedings on 34th Annual ACM Symposium on Theory of Computing, May 19-21, 2002, Montréal, Québec, Canada* (2002), J. H. Reif, Ed., ACM, pp. 494–503.
- [45] CANETTI, R., AND RABIN, T. Fast asynchronous byzantine agreement with optimal resilience. In *Proceedings of the Twenty-Fifth Annual ACM Symposium on Theory of Computing, May 16-18, 1993, San Diego, CA, USA* (1993), S. R. Kosaraju, D. S. Johnson, and A. Aggarwal, Eds., ACM, pp. 42–51.
- [46] CANETTI, R., AND RABIN, T. Universal composition with joint state. In *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings* (2003), D. Boneh, Ed., vol. 2729 of *Lecture Notes in Computer Science*, Springer, pp. 265–281.
- [47] CANETTI, R., SARKAR, P., AND WANG, X. Triply adaptive UC NIZK. In *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Public-Key Cryptography, Røros, Norway, May 12-15, 2022, Proceedings, Part II* (2022), S. Agrawal and D. Lin, Eds., vol. 13792 of *Lecture Notes in Computer Science*, Springer, pp. 466–495.
- [48] CHEN, J., AND MICALI, S. Algorand: A secure and efficient distributed ledger. *Theor. Comput. Sci.* 777 (2019), 155–183.
- [49] CHEN, M., DEY, P., GANESH, C., MUKHERJEE, P., SARKAR, P., AND SASMAL, S. Universally composable non-interactive zero-knowledge from sigma protocols via a new straight-line compiler. In *Public-Key Cryptography - PKC 2025 - 28th IACR International Conference on Practice and Theory of Public-Key Cryptography, Røros, Norway, May 12-15, 2025, Proceedings, Part I* (2025), T. Jager and J. Pan, Eds., vol. 15674 of *Lecture Notes in Computer Science*, Springer, pp. 381–417.
- [50] CHEVALLIER-MAMES, B., PAILLIER, P., AND POINTCHEVAL, D. Encoding-free elgamal encryption without random oracles. In *Public Key Cryptography - PKC 2006, 9th International Conference on Theory and Practice of Public-Key Cryptography, New York, NY, USA, April 24-26, 2006, Proceedings* (2006), M. Yung, Y. Dodis, A. Kiayias, and T. Malkin, Eds., vol. 3958 of *Lecture Notes in Computer Science*, Springer, pp. 91–104.
- [51] CHIESA, A., AND FENZI, G. zkSNARKs in the ROM with unconditional UC-security. In *Theory of Cryptography - 22nd International Conference, TCC 2024, Milan, Italy, December 2-6, 2024, Proceedings, Part I* (2024), E. Boyle and M. Mahmoody, Eds., vol. 15364 of *Lecture Notes in Computer Science*, Springer, pp. 67–89.
- [52] CIAMPI, M., RAVI, D., SINISCALCHI, L., AND WALDNER, H. Round-optimal multi-party computation with identifiable abort. In *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part I* (2022), O. Dunkelman and S. Dziembowski, Eds., vol. 13275 of *Lecture Notes in Computer Science*, Springer, pp. 335–364.
- [53] CIVIT, P., COLLINS, D., GRAMOLI, V., GUERRAOU, R., KOMATOVIC, J., VIDIGUEIRA, M., AND ZARBAFIAN, P. Scalable accountable byzantine agreement and beyond. In *IEEE Symposium on Security and Privacy, SP 2026 (To appear), San Francisco, CA, USA, May 18-20, 2026* (2026), IEEE.
- [54] CIVIT, P., GILBERT, S., AND GRAMOLI, V. Polygraph: Accountable byzantine agreement. In *41st IEEE International Conference on Distributed Computing Systems, ICDCS 2021, Washington DC, USA, July 7-10, 2021* (2021), IEEE, pp. 403–413.
- [55] CIVIT, P., GILBERT, S., GRAMOLI, V., GUERRAOU, R., AND KOMATOVIC, J. As easy as ABC: optimal (a)ccountable (b)yzantine (c)onsensus is easy! In *2022 IEEE International Parallel and Distributed Processing Symposium, IPDPS 2022, Lyon, France, May 30 - June 3, 2022* (2022), IEEE, pp. 560–570.
- [56] CIVIT, P., GILBERT, S., GRAMOLI, V., GUERRAOU, R., AND KOMATOVIC, J. As easy as ABC: optimal (a)ccountable (b)yzantine (c)onsensus is easy! *J. Parallel Distributed Comput.* 181 (2023), 104743.
- [57] CIVIT, P., GILBERT, S., GRAMOLI, V., GUERRAOU, R., KOMATOVIC, J., MILOSEVIC, Z., AND SEREDINSCHI, A. Crime and Punishment in Distributed Byzantine Decision Tasks. In *42nd IEEE International Conference on Distributed Computing Systems, ICDCS 2022, Bologna, Italy, July 10-13, 2022* (2022), IEEE, pp. 34–44.
- [58] CLEMENT, A., JUNQUEIRA, F., KATE, A., AND RODRIGUES, R. On the (limited) power of non-equivocation. In *Proceedings of the 2012 ACM Symposium on Principles of Distributed Computing* (New York, NY, USA, 2012), PODC ’12, Association for Computing Machinery, p. 301–308.
- [59] CLEVE, R. Limits on the security of coin flips when half the processors are faulty (extended abstract). In *Proceedings of the 18th Annual ACM Symposium on Theory of Computing, May 28-30, 1986, Berkeley, California, USA* (1986), J. Hartmanis, Ed., ACM, pp. 364–369.

- [60] COAN, B. A. A compiler that increases the fault tolerance of asynchronous protocols. *IEEE Trans. Computers* 37, 12 (1988), 1541–1553.
- [61] COHEN, R. Asynchronous secure multiparty computation in constant time. In *Public-Key Cryptography - PKC 2016 - 19th IACR International Conference on Practice and Theory in Public-Key Cryptography, Taipei, Taiwan, March 6-9, 2016, Proceedings, Part II* (2016), C. Cheng, K. Chung, G. Persiano, and B. Yang, Eds., vol. 9615 of *Lecture Notes in Computer Science*, Springer, pp. 183–207.
- [62] COHEN, R., DOERNER, J., KONDI, Y., AND SHELAT, A. Secure multiparty computation with identifiable abort via vindicating release. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VIII* (2024), L. Reyzin and D. Stebila, Eds., vol. 14927 of *Lecture Notes in Computer Science*, Springer, pp. 36–73.
- [63] COHEN, R., AND LINDELL, Y. Fairness versus guaranteed output delivery in secure multiparty computation. In *Advances in Cryptology - ASIACRYPT 2014 - 20th International Conference on the Theory and Application of Cryptology and Information Security, Kaoshiung, Taiwan, R.O.C., December 7-11, 2014, Proceedings, Part II* (2014), P. Sarkar and T. Iwata, Eds., vol. 8874 of *Lecture Notes in Computer Science*, Springer, pp. 466–485.
- [64] COHEN, R., AND LINDELL, Y. Fairness versus guaranteed output delivery in secure multiparty computation. *J. Cryptol.* 30, 4 (2017), 1157–1186.
- [65] COLLINS, D., GUERRAOU, R., KOMATOVIC, J., KUZNETSOV, P., MONTI, M., PAVLOVIC, M., PIGNOLET, Y.-A., SEREDINSCHI, D.-A., TONKIKH, A., AND XYGKIS, A. Online payments by merely broadcasting messages. In *2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)* (2020), IEEE, pp. 26–38.
- [66] CORREIA, M., VERONESE, G. S., AND LUNG, L. C. Asynchronous byzantine consensus with $2f+1$ processes. In *Proceedings of the 2010 ACM Symposium on Applied Computing (SAC), Sierre, Switzerland, March 22-26, 2010* (2010), S. Y. Shin, S. Ossowski, M. Schumacher, M. J. Palakal, and C. Hung, Eds., ACM, pp. 475–480.
- [67] CRAMER, R., DAMGÅRD, I., AND NIELSEN, J. B. *Secure Multiparty Computation and Secret Sharing*. Cambridge University Press, 2015.
- [68] CUNNINGHAM, R. K., FULLER, B., AND YAKOUBOV, S. Catching MPC cheaters: Identification and openability. In *Information Theoretic Security - 10th International Conference, ICITS 2017, Hong Kong, China, November 29 - December 2, 2017, Proceedings* (2017), J. Shikata, Ed., vol. 10681 of *Lecture Notes in Computer Science*, Springer, pp. 110–134.
- [69] DAS, S., DUAN, S., LIU, S., MOMOSE, A., REN, L., AND SHOUP, V. Asynchronous consensus without trusted setup or public-key cryptography. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024* (2024), B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie, Eds., ACM, pp. 3242–3256.
- [70] DAS, S., AND REN, L. Adaptively secure BLS threshold signatures from DDH and co-cdh. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VII* (2024), L. Reyzin and D. Stebila, Eds., vol. 14926 of *Lecture Notes in Computer Science*, Springer, pp. 251–284.
- [71] DAS, S., XIANG, Z., TOMESCU, A., SPIEGELMAN, A., PINKAS, B., AND REN, L. Verifiable secret sharing simplified. In *IEEE Symposium on Security and Privacy, SP 2025, San Francisco, CA, USA, May 12-15, 2025* (2025), M. Blanton, W. Enck, and C. Nita-Rotaru, Eds., IEEE, pp. 633–651.
- [72] DE SOUZA, L. F., KUZNETSOV, P., RIEUTORD, T., AND TUCCI PIERGIOVANNI, S. Accountability and reconfiguration: Self-healing lattice agreement. In *25th International Conference on Principles of Distributed Systems, OPODIS 2021, December 13-15, 2021, Strasbourg, France* (2021), Q. Bramas, V. Gramoli, and A. Milani, Eds., vol. 217 of *LIPICs*, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, pp. 25:1–25:23.
- [73] DE SOUZA, L. F., KUZNETSOV, P., AND TONKIKH, A. Distributed randomness from approximate agreement. In *36th International Symposium on Distributed Computing, DISC 2022, October 25-27, 2022, Augusta, Georgia, USA* (2022), C. Scheideler, Ed., vol. 246 of *LIPICs*, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, pp. 24:1–24:21.
- [74] DWORK, C., LYNCH, N., AND STOCKMEYER, L. Consensus in the presence of partial synchrony. *Journal of the Association for Computing Machinery*, Vol. 35, No. 2, pp.288-323 (1988).
- [75] EVANS, D., KOLESNIKOV, V., AND ROSULEK, M. A pragmatic introduction to secure multi-party computation. *Found. Trends Priv. Secur.* 2, 2-3 (2018), 70–246.
- [76] FLEISCHHACKER, N., HALL-ANDERSEN, M., SIMKIN, M., AND WAGNER, B. Jackpot: Non-interactive aggregatable lotteries. In *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part VI* (2024), K. Chung and Y. Sasaki, Eds., vol. 15489 of *Lecture Notes in Computer Science*, Springer, pp. 365–397.
- [77] FREILING, F. C., GUERRAOU, R., AND KUZNETSOV, P. The failure detector abstraction. *ACM Comput. Surv.* 43, 2 (2011), 9:1–9:40.
- [78] FUJISAKI, E. Improving practical uc-secure commitments based on the DDH assumption. *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.* 105-A, 3 (2022), 182–194.

- [79] GANESH, C., KONDI, Y., ORLANDI, C., PANCHOLI, M., TAKAHASHI, A., AND TSCHUDI, D. Witness-succinct universally-composable snarks. In *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part II* (2023), C. Hazay and M. Stam, Eds., vol. 14005 of *Lecture Notes in Computer Science*, Springer, pp. 315–346.
- [80] GARAY, J. A., KIAYIAS, A., AND LEONARDOS, N. The bitcoin backbone protocol: Analysis and applications. In *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part II* (2015), E. Oswald and M. Fischlin, Eds., vol. 9057 of *Lecture Notes in Computer Science*, Springer, pp. 281–310.
- [81] GARAY, J. A., KIAYIAS, A., AND LEONARDOS, N. The bitcoin backbone protocol: Analysis and applications. *J. ACM* 71, 4 (2024), 25:1–25:49.
- [82] GOLDBREICH, O. *The Foundations of Cryptography - Volume 1: Basic Techniques*. Cambridge University Press, 2001.
- [83] GOLDBREICH, O., MICALI, S., AND WIGDERSON, A. How to play any mental game or A completeness theorem for protocols with honest majority. In *Proceedings of the 19th Annual ACM Symposium on Theory of Computing, 1987, New York, New York, USA* (1987), A. V. Aho, Ed., ACM, pp. 218–229.
- [84] GOLDBREICH, O., MICALI, S., AND WIGDERSON, A. How to play any mental game, or a completeness theorem for protocols with honest majority. In *Providing Sound Foundations for Cryptography: On the Work of Shafi Goldwasser and Silvio Micali*, O. Goldreich, Ed. ACM, 2019, pp. 307–328.
- [85] GOLDWASSER, S., MICALI, S., AND RACKOFF, C. The knowledge complexity of interactive proof-systems (extended abstract). In *Proceedings of the 17th Annual ACM Symposium on Theory of Computing, May 6-8, 1985, Providence, Rhode Island, USA* (1985), R. Sedgewick, Ed., ACM, pp. 291–304.
- [86] GRAF, M., KÜSTERS, R., AND RAUSCH, D. AUC: accountable universal composability. In *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023* (2023), IEEE, pp. 1148–1167.
- [87] GRAF, M., KÜSTERS, R., RAUSCH, D., EGGER, S., BECHTOLD, M., AND FLINSPACH, M. Accountable bulletin boards: Definition and provably secure implementation. *IACR Cryptol. ePrint Arch.* (2023), 1869.
- [88] GROTH, J., AND SHOUP, V. Fast batched asynchronous distributed key generation. In *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part V* (2024), M. Joye and G. Leander, Eds., vol. 14655 of *Lecture Notes in Computer Science*, Springer, pp. 370–400.
- [89] GUERRAUI, R., KUZNETSOV, P., MONTI, M., PAVLOVIĆ, M., AND SEREDINSCHI, D.-A. The consensus number of a cryptocurrency. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing* (2019), pp. 307–316.
- [90] HAEBERLEN, A., KOUZNETSOV, P., AND DRUSCHEL, P. Peerreview: practical accountability for distributed systems. In *Proceedings of the 21st ACM Symposium on Operating Systems Principles 2007, SOSP 2007, Stevenson, Washington, USA, October 14-17, 2007* (2007), T. C. Bressoud and M. F. Kaashoek, Eds., ACM, pp. 175–188.
- [91] HIRT, M., NIELSEN, J. B., AND PRZYDATEK, B. Asynchronous multi-party computation with quadratic communication. In *Automata, Languages and Programming, 35th International Colloquium, ICALP 2008, Reykjavik, Iceland, July 7-11, 2008, Proceedings, Part II - Track B: Logic, Semantics, and Theory of Programming & Track C: Security and Cryptography Foundations* (2008), L. Aceto, I. Damgård, L. A. Goldberg, M. M. Halldórsson, A. Ingólfssdóttir, and I. Walukiewicz, Eds., vol. 5126 of *Lecture Notes in Computer Science*, Springer, pp. 473–485.
- [92] HO, C., DOLEV, D., AND VAN RENESSE, R. Making distributed applications robust. In *Principles of Distributed Systems, 11th International Conference, OPODIS 2007, Guadeloupe, French West Indies, December 17-20, 2007. Proceedings* (2007), E. Tovar, P. Tsigas, and H. Fouchal, Eds., vol. 4878 of *Lecture Notes in Computer Science*, Springer, pp. 232–246.
- [93] HO, C., VAN RENESSE, R., BICKFORD, M., AND DOLEV, D. Nysiad: Practical protocol transformation to tolerate byzantine failures. In *5th USENIX Symposium on Networked Systems Design & Implementation, NSDI 2008, April 16-18, 2008, San Francisco, CA, USA, Proceedings* (2008), J. Crowcroft and M. Dahlin, Eds., USENIX Association, pp. 175–188.
- [94] ISHAI, Y., OSTROVSKY, R., AND ZIKAS, V. Secure multi-party computation with identifiable abort. In *Advances in Cryptology - CRYPTO 2014 - 34th Annual Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2014, Proceedings, Part II* (2014), J. A. Garay and R. Gennaro, Eds., vol. 8617 of *Lecture Notes in Computer Science*, Springer, pp. 369–386.
- [95] KAMP, S. H., AND NIELSEN, J. B. Byzantine agreement decomposed: Honest majority asynchronous total-order broadcast from reliable broadcast. *IACR Cryptol. ePrint Arch.* (2023), 1738.
- [96] KIAYIAS, A., KULDMAA, A., LIPMAA, H., SIIM, J., AND ZACHARIAS, T. On the security properties of e-voting bulletin boards. In *Security and Cryptography for Networks - 11th International Conference, SCN 2018, Amalfi, Italy, September 5-7, 2018, Proceedings* (2018), D. Catalano and R. D. Prisco, Eds., vol. 11035 of *Lecture Notes in Computer Science*, Springer, pp. 505–523.
- [97] KÜSTERS, R., TRUDERUNG, T., AND VOGT, A. Accountability: definition and relationship to verifiability. In *Proceedings of the 17th ACM Conference on Computer and Communications Security, CCS 2010, Chicago, Illinois, USA, October 4-8, 2010* (2010), E. Al-Shaer, A. D. Keromytis, and V. Shmatikov, Eds., ACM, pp. 526–535.

- [98] KÜSTERS, R., TUENGERTHAL, M., AND RAUSCH, D. The IITM model: A simple and expressive model for universal composability. *J. Cryptol.* 33, 4 (2020), 1461–1584.
- [99] LEINWEBER, M., AND HARTENSTEIN, H. Brief announcement: Let it TEE: asynchronous byzantine atomic broadcast with $n \geq 2f+1$. In *37th International Symposium on Distributed Computing, DISC 2023, October 10–12, 2023, L'Aquila, Italy* (2023), R. Oshman, Ed., vol. 281 of *LIPIcs*, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, pp. 43:1–43:7.
- [100] LINCOLN, P., MITCHELL, J. C., MITCHELL, M., AND SCEDROV, A. A probabilistic poly-time framework for protocol analysis. In *CCS '98, Proceedings of the 5th ACM Conference on Computer and Communications Security, San Francisco, CA, USA, November 3–5, 1998* (1998), L. Gong and M. K. Reiter, Eds., ACM, pp. 112–121.
- [101] LYNCH, N. A. *Distributed Algorithms*. Elsevier, 1996.
- [102] LYSYANSKAYA, A., AND ROSENBLUM, L. N. Efficient and universally composable non-interactive zero-knowledge proofs of knowledge with security against adaptive corruptions. *IACR Cryptol. ePrint Arch.* (2022), 1484.
- [103] MATT, C., NIELSEN, J. B., AND THOMSEN, S. E. Formalizing delayed adaptive corruptions and the security of flooding networks. In *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15–18, 2022, Proceedings, Part II* (2022), Y. Dodis and T. Shrimpton, Eds., vol. 13508 of *Lecture Notes in Computer Science*, Springer, pp. 400–430.
- [104] MICALI, S. Computationally sound proofs. *SIAM J. Comput.* 30, 4 (2000), 1253–1298.
- [105] NAYAK, K. Player Replaceability - Towards Adaptive Security and Sub-quadratic Communication Simultaneously (Part I), 2023. <https://decentralizedthoughts.github.io/2023-01-05-player-replaceability-I/>.
- [106] NAYAK, K. Player Replaceability - Towards Adaptive Security and Sub-quadratic Communication Simultaneously (Part II), 2023. <https://decentralizedthoughts.github.io/2023-01-05-player-replaceability-II/>.
- [107] PEDERSEN, T. P. Non-interactive and information-theoretic secure verifiable secret sharing. In *Advances in Cryptology - CRYPTO '91, 11th Annual International Cryptology Conference, Santa Barbara, California, USA, August 11–15, 1991, Proceedings* (1991), J. Feigenbaum, Ed., vol. 576 of *Lecture Notes in Computer Science*, Springer, pp. 129–140.
- [108] PFITZMANN, B., AND WAIDNER, M. Composition and integrity preservation of secure reactive systems. In *CCS 2000, Proceedings of the 7th ACM Conference on Computer and Communications Security, Athens, Greece, November 1–4, 2000* (2000), D. Gritzalis, S. Jajodia, and P. Samarati, Eds., ACM, pp. 245–254.
- [109] RAMANATHAN, A., MITCHELL, J. C., SCEDROV, A., AND TEAGUE, V. Probabilistic bisimulation and equivalence for security analysis of network protocols. In *Foundations of Software Science and Computation Structures, 7th International Conference, FOSSACS 2004, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2004, Barcelona, Spain, March 29 - April 2, 2004, Proceedings* (2004), I. Walukiewicz, Ed., vol. 2987 of *Lecture Notes in Computer Science*, Springer, pp. 468–483.
- [110] RAMBAUD, M. Adaptively secure consensus with linear complexity and constant round under honest majority in the bare PKI model, and separation bounds from the idealized message-authentication model. *IACR Cryptol. ePrint Arch.* (2023), 1757.
- [111] RAUSCH, D., KÜSTERS, R., AND CHEVALIER, C. Embedding the UC model into the IITM model. In *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part II* (2022), O. Dunkelman and S. Dziembowski, Eds., vol. 13276 of *Lecture Notes in Computer Science*, Springer, pp. 242–272.
- [112] RIVINIUS, M., REISERT, P., RAUSCH, D., AND KÜSTERS, R. Publicly accountable robust multi-party computation. In *43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22–26, 2022* (2022), IEEE, pp. 2430–2449.
- [113] SHOUP, V. A theoretical take on a practical consensus protocol. *Cryptology ePrint Archive* (2024).
- [114] SHOUP, V., AND SMART, N. P. Lightweight asynchronous verifiable secret sharing with optimal resilience. *J. Cryptol.* 37, 3 (2024), 27.
- [115] SPINI, G., AND FEHR, S. Cheater detection in SPDZ multiparty computation. In *Information Theoretic Security - 9th International Conference, ICITS 2016, Tacoma, WA, USA, August 9–12, 2016, Revised Selected Papers* (2016), A. C. A. Nascimento and P. S. L. M. Barreto, Eds., vol. 10015 of *Lecture Notes in Computer Science*, pp. 151–176.